



الجمهورية العربية السورية
المعهد العالي للعلوم التطبيقية والتكنولوجيا
قسم المعلومات
العام الدراسي 2025/2026

تصميم جدار نار قابل للبرمجة باستخدام تقنية الشبكات المعرّفة برمجياً (SDN) Software-Defined Networking

تقديم
محمد غيث فارس الأطرش

إشراف
د. حسين خيو
ما. محمد حمدان

16/08/2026

الملخص

في ظل التزايد المستمر في تعقيد الهجمات السيبرانية والقيود التي تفرضها جدران النار التقليدية من حيث المرونة وسهولة الإدارة، يهدف هذا المشروع إلى تصميم وتنفيذ جدار نار قابل للبرمجة اعتماداً على مفهوم الشبكات المعرفة برمجياً SDN. يعتمد النظام على فصل مستوي التحكم (Control Plane) عن مستوي تمرير البيانات (Data Plane)، مما يتيح إدارة مركزية لحركة السير (Traffic) وتطبيق سياسات أمنية ديناميكية وقابلة للتحديث دون الحاجة إلى إعادة تهيئة مكونات الشبكة بشكل يدوي. تم تصميم جدار النار بحيث يقوم بتحليل الرزم الشبكية واتخاذ قرارات السماح أو الحظر وفق قواعد أمنية قابلة للبرمجة، مع الاستفادة من متحكم SDN لإدارة تدفق البيانات بين مكونات الشبكة. كما تم اختبار النظام في بيئة محاكاة للتحقق من قدرته على اكتشاف ومنع حركة السير غير المصرح بها، مع الحفاظ على الأداء والكفاءة في إدارة الشبكة. أظهرت نتائج الاختبارات أن استخدام تقنيات SDN يساهم في تحسين مرونة إدارة السياسات الأمنية، وتبسيط عمليات الإدارة، وتعزيز القدرة على التكيف مع التهديدات الأمنية الحديثة مقارنةً بالحلول التقليدية.

Abstract

With the increasing complexity of cyber threats and the limitations of traditional firewalls in terms of flexibility and manageability, this project aims to design and implement a programmable firewall based on Software-Defined Networking (SDN). The proposed system leverages the separation of the control plane from the data plane to provide centralized traffic management and dynamic security policy enforcement without requiring manual reconfiguration of network devices. The firewall was designed to inspect network packets and make forwarding or blocking decisions according to programmable security rules, while utilizing an SDN controller to manage traffic flows across the network. The system was evaluated in a simulated environment to verify its ability to detect and prevent unauthorized traffic while maintaining network performance and operational efficiency. Experimental results demonstrate that the SDN-based firewall improves the flexibility of security policy management, simplifies network administration, and enhances adaptability to modern cybersecurity threats compared with conventional firewall solutions.

المحتويات

ix	الاختصارات
1	الفصل الأول
1	التعريف بالمشروع
1	يتضمن هذا الفصل التعريف بالمشروع ومتطلباته
1	1.1- مقدمة
2	2.1- هدف المشروع
2	3.1- المتطلبات الوظيفية
3	4.1- المتطلبات غير الوظيفية
4	الفصل الثاني
4	الدراسة النظرية
4	1.2- الشبكات المعرفة برمجياً (Software-Defined Networks)
4	1.1.2- تعريف
4	2.1.2- مزايا SDN
5	3.1.2- معمارية SDN
6	2.2- ميفاق OpenFlow
6	1.2.2- تعريف
6	2.2.2- جداول التدفق (Flow Tables)
6	3.2.2- مفهومي Packet-In و Flow-Mod
6	4.2.2- الأولويات في قواعد التدفق
7	3.2- جدار النار (Firewall)
7	1.3.2- تعريف
7	2.3.2- أنواع جدران النار

7	3.3.2- آلية السماح والحجب
8	4.2- متحكمات SDN
8	5.2- REST API
9	6.2- ميفيق الشبكة المدعومة
9	7.2- قواعد المعطيات (Databases)
9	8.2- تعلُّم الآلة والتعلُّم العميق (Machine Learning/Deep Learning)
11	الفصل الثالث
11	الدراسة المرجعية
11	1.3- مقدمة
11	2.3- أداء متحكمات SDN
12	3.3- جدران النار المبنية على SDN
12	4.3- التطبيق الديناميكي لقواعد التدفق
13	5.3- موقع المشروع الحالي ومساهمته المقترحة
14	الفصل الرابع
14	الدراسة التحليلية
14	1.4- تحديد الفاعلين (Actors)
16	3.4- السرد النصي لحالات الاستخدام
16	1.3.4- إدارة سياسات جدار النار
17	2.3.4- عرض لوحة التحكم
19	3.3.4- عرض التقارير الأمنية
20	4.3.4- عرض السجلات
22	الفصل الخامس
22	المنهجية المقترحة
22	1.5- مقدمة

22	2.5- النموذج المقترح.....
23	1.2.5- منهجية طبقة التحكم (Controller Layer)
25	2.2.5- منهجية طبقة التطبيق (Application Layer).....
28	3.5- آلية التكامل بين الطبقات ومنطق تطبيق القواعد
29	الفصل السادس
29	تصميم النظام
29	1.6- مقدمة
29	2.6- تصميم مكونات النظام
29	1.2.6- نسيئة المتحكم (Controller Module)
31	2.2.6- نسيئة إدارة القواعد (Rules Management Module)
32	3.2.6- نسيئة مراقبة التدفق (Flow Monitoring Module)
33	4.2.6- نسيئة الواجهة (Interface Module).....
35	3.6- تصميم قاعدة المعطيات
35	1.3.6- مخطط الكيانات والعلاقات (Entity Relationship Diagram)
37	4.6- مخطط الصفوف (Class Diagram).....
40	الفصل السابع
40	الأدوات المستخدمة
40	1.7- متحكم Ryu
40	2.7- Open vSwitch (OVS)
40	3.7- لغة البرمجة Python
40	4.7- إطار العمل Flask
41	5.7- HTML5, CSS3, JavaScript
41	6.7- Scalable Vector Graphics (SVG)
41	7.7- Containernet

41	 Docker -8.7
41	 SQLite -9.7
42	 بيئة التطوير -10.7
43	 الفصل الثامن
43	 تنجيز النظام
43	 1.8- مقدمة
43	 2.8- تنجيز نَسِيْقَة المتحكم
43	 1.2.8- مقدمة
43	 2.2.8- تفاصيل التنجيز
44	 3.8- تنجيز نَسِيْقَة إدارة القواعد
44	 1.3.8- مقدمة
44	 2.3.8- تفاصيل التنجيز
45	 4.8- تنجيز نَسِيْقَة مراقبة التدفق
45	 1.4.8- مقدمة
45	 2.4.8- تفاصيل التنجيز
46	 5.8- تنجيز نَسِيْقَة الواجهة
46	 1.5.8- مقدمة
46	 2.5.8- تفاصيل التنجيز
46	 6.8- تنجيز قاعدة المعطيات
46	 1.6.8- مقدمة
47	 2.6.8- تفاصيل التنجيز
48	 7.8- مخطط النظام التنجيزي
49	 8.8- تقييم خوارزميات تعلُّم الآلة لتصنيف الهجمات
49	 1.8.8- مجموعة البيانات (Dataset)

49	2.8.8- المعالجة المسبقة للبيانات (Preprocessing) وهندسة السمات (Feature Engineering)
52	3.8.8- الخوارزميات المستخدمة
55	4.8.8- منهجية التدريب والاختبار
56	5.8.8- معايير تقييم الأداء
56	6.8.8- تحليل النتائج ومناقشتها
59	7.8.8- اختيار الخوارزمية الأنسب
60	الفصل التاسع
60	اختبارات النظام ومناقشة النتائج
60	يوضح هذا الفصل الاختبارات التي تم تطبيقها للتأكد من تلبية النظام للمتطلبات.
60	1.9- اختبار الوحدات (Units Testing)
61	2.9- اختبار التكامل (Integration Testing)
62	3.9- الاختبارات الأمنية (Security Testing)
64	4.9- اختبارات الحمل (Load Testing)
65	5.9- اختبارات الإجهاد (Stress Testing)
66	6.9- اختبارات الأداء (Performance Testing)
68	7.9- تغطية المتطلبات الوظيفية من خلال مكونات النظام
69	الخاتمة والآفاق المستقبلية
72	المراجع

قائمة الأشكال

الشكل 1- معمارية الشبكات المعرفة برمجياً.....	5
الشكل 2- مبدأ عمل جدار النار.....	8
الشكل 3- مخطط حالات الاستخدام.....	15
الشكل 4- معمارية النظام المقترح.....	23
الشكل 5- سير عملية إضافة قاعدة وتطبيقها في النظام.....	26
الشكل 6- سير عملية حذف قاعدة موجودة.....	26
الشكل 7- سير عملية التعديل على قاعدة موجودة.....	27
الشكل 8- آلية تفاعل مبدلات OpenFlow مع متحكم Ryu.....	30
الشكل 9- سير عملية المراقبة (Monitoring).....	32
الشكل 10- مخطط الكيانات والعلاقات.....	36
الشكل 11- مخطط الصفوف للنظام.....	38
الشكل 12- التكامل بين مكونات النظام.....	48
الشكل 13- توزيع أنواع الهجمات في مجموعة البيانات.....	49
الشكل 14- أعلى 20 سمة ارتباطاً بالتصنيف هجمة.....	51
الشكل 15- توزيع أفضل 6 سمات (سليم مقابل هجوم).....	52
الشكل 16- توزيع الفئات على تقسيم 60/20/20.....	55
الشكل 17- منحنيات تدريب خوارزميات التعلم العميق.....	58
الشكل 18- نتائج اختبارات الوحدات.....	61
الشكل 19- نتائج اختبارات الوحدات (تكملة).....	61
الشكل 20- نتائج اختبارات التكامل.....	62
الشكل 21- نتائج الاختبارات الأمنية.....	64
الشكل 22- نتائج اختبارات الحمل.....	65
الشكل 23- نتائج اختبارات الإجهاد.....	66
الشكل 24- نتائج اختبارات الأداء.....	67

قائمة الجداول

الجدول 1- إدارة سياسات جدار النار.....	16
الجدول 2- السيناريو الأساسي الناجح لحالة استخدام إدارة سياسات جدار النار.....	16
الجدول 3- عرض لوحة التحكم.....	17
الجدول 4- السيناريو الأساسي الناجح لحالة استخدام عرض لوحة التحكم.....	18
الجدول 5- عرض التقارير الأمنية.....	19
الجدول 6- السيناريو الأساسي الناجح لحالة استخدام عرض التقارير الأمنية.....	19
الجدول 7- عرض السجلات.....	20
الجدول 8- السيناريو الأساسي الناجح لحالة استخدام عرض السجلات.....	20
الجدول 9- نظام الأولويات في القواعد.....	31
الجدول 10- نتائج خوارزميات التدريب.....	57
الجدول 11- المتطلبات الوظيفية المنجزة.....	68

الاختصارات

API	Application Programming Interface (واجهة برمجة التطبيقات)
AUC	Area Under the Curve (المساحة تحت المنحني)
Cbench	Controller Benchmark (أداة قياس أداء المتحكم)
CNN-1D	1-Dimensional Convolutional Neural Networks (الشبكات العصبونية التلافيفية أحادية البعد)
CSS	Cascading Style Sheets (ورقيات الطراز المتشلسلة)
CSV	Comma Separated Values (ملف القيم المفصولة بفواصل)
DDoS	Distributed Denial of Service (هجوم منع الخدمة الموزع)
DMZ	DeMilitarized Zone (منطقة عازلة)
DoS	Denial of Service (هجوم منع الخدمة)
HTML	HyperText Markup Language (لغة تأشير نص ترابطي)
HTTP	HyperText Transfer Protocol (ميثاق نقل النصوص الترابطية)
ICMP	Internet Control Message Protocol (ميثاق رسالة التحكم في إنترنت)
IDS	Intrusion Detection System (نظام كشف التسلل)
IP	Internet Protocol (ميثاق الإنترنت)
IPv4	Internet Protocol version 4 (ميثاق الإنترنت الإصدار الرابع)

IPv6	Internet Protocol version 6 (ميثاق الإنترنت الإصدار السادس)
JSON	JavaScript Object Notation (ترميز كائنات جافا سكريبت)
LAN	Local Area Network (شبكة محلية)
LightGBM	Light Gradient Boosting Machine (آلة التعزيز المتدرج الخفيفة)
LSTM	Long Short-Term Memory (شبكة الذاكرة طويلة وقصير الأمد)
OVS	Open vSwitch (مبدلة افتراضية مفتوحة المصدر)
RAM	Random Access Memory (ذاكرة الوصول العشوائي)
ReLU	Rectified Linear Unit (وحدة التنشيط الخطي المصحح)
REST	REpresentational State Transfer (نقل الحالة التمثيلي)
ROC	Receiver Operating Characteristic (خاصية تشغيل المستقبل)
SDFW	SDN-based Stateful Distributed Firewall (جدار نار موزع برقابة الحالة مبني على الشبكات المعرفة برمجياً)
SDN	Software-Defined Networking (الشبكات المعرفة برمجياً)
SIEM	Security Information and Event Management (إدارة معلومات وأحداث الأمن)
SQL	Structured Query Language (لغة استفسار بنيوية)
SQLite	Structures Query Language Lite (نظام إدارة قواعد معطيات علائقية خفيف الوزن)
SVG	Scalable Vector Graphics (صفيفة بيانيات فيديوية)
SYN	Synchronize (مزامنة)

TCP	Transmission Control Protocol (ميفاق التحكم في الإرسال)
UDP	User Datagram Protocol (ميفاق برقية معطيات المستخدم)
vCPU	Virtual Central Processing Unit (وحدة معالجة مركزية افتراضية)
XSS	Cross-Site Scripting (البرمجة النصية عبر المواقع)
XGBoost	eXtreme Gradient Boosting (التعزيز المتدرج المتطرف)

الفصل الأول

التعريف بالمشروع

يتضمن هذا الفصل التعريف بالمشروع ومتطلباته.

1.1- مقدمة

في ظل التطور المتسارع لتقنيات الاتصالات والشبكات، والاعتماد المتزايد على الخدمات الرقمية والحوسبة السحابية (Cloud Computing)، أصبحت الشبكات الحاسوبية هدفاً رئيسياً للهجمات السيبرانية التي تتنوع بين محاولات الوصول غير المصرح به، وهجمات حجب الخدمة، وانتشار البرمجيات الخبيثة، وقد أدى هذا الواقع إلى زيادة الحاجة إلى حلول أمنية متقدمة قادرة على حماية البنية التحتية للشبكات مع الحفاظ على كفاءتها ومرونتها [1]، وتُعد جدران النار (Firewalls) من أهم وسائل الدفاع في أمن الشبكات، حيث تقوم بمراقبة حركة البيانات والتحكم بها وفق سياسات أمنية محددة [2]، إلا أنَّ جدران النار التقليدية ما تزال تعاني من محدودية في المرونة وصعوبة في إدارة السياسات الأمنية وتحديثها، خاصةً في البيئات الشبكية الكبيرة [3].

مع ظهور مفهوم الشبكات المعرفة برمجياً، أصبح بالإمكان فصل مستوى التحكم عن مستوى تمرير البيانات، مما أتاح إدارة مركزية للشبكة وإعادة تهيئة سياساتها بصورة ديناميكية. توفر هذه البنية بيئة مناسبة لتطوير حلول أمنية أكثر ذكاءً وقابلية للتوسع، من خلال تطبيق السياسات الأمنية بشكل مركزي والاستجابة السريعة للتهديدات والتغيرات في حركة السير الشبكية، مع تقليل التعقيد الإداري وتحسين كفاءة إدارة الشبكات [3].

2.1- هدف المشروع

يتمثل الهدف الرئيس للمشروع في تصميم وتطوير جدار نار قابل للبرمجة يعتمد على تقنية SDN، بهدف تعزيز أمن الشبكات من خلال تطبيق سياسات أمنية ديناميكية وقابلة للتحديث بصورة مركزية. كما يهدف إلى الاستفادة من الإمكانيات التي توفرها بنية SDN في تبسيط إدارة الشبكات، وتحسين قابلية التوسع، وزيادة سرعة الاستجابة للتهديدات الأمنية والتغيرات في حركة البيانات. يتم ذلك من خلال تطوير نموذج لجدار نار قادر على مراقبة الرزم الشبكية وتحليلها وتطبيق السياسات الأمنية بشكل مركزي، مع تقييم أدائه وفاعليته في بيئة محاكاة تمثل سيناريوهات تشغيل واقعية، بما يوضح مزايا جدران النار القابلة للبرمجة مقارنةً بالحلول التقليدية.

3.1- المتطلبات الوظيفية

1. يجب أن يسمح النظام للمستخدم بإدارة قواعد جدار النار عبر واجهة وب. يشمل ذلك ما يأتي:
 - 1.1. إضافة قاعدة جديدة.
 - 2.1. تعديل قاعدة موجودة.
 - 3.1. حذف قاعدة موجودة.
 - 4.1. عرض جميع القواعد.
2. يجب أن يقوم النظام بتطبيق القواعد الأمنية على التدفقات الشبكية باستخدام متحكم SDN.
 - 1.2. تحويل القواعد إلى OpenFlow Entries.
 - 2.2. تثبيت القواعد على OpenFlow Switches.
 - 3.2. منع أو السماح بالتدفقات حسب السياسة.
3. يجب أن يقوم النظام بتعديل القواعد أثناء تشغيل النظام دون إعادة تشغيل المتحكم أو الشبكة.
4. يجب أن يوفر النظام معلومات عن حالة الشبكة والتدفقات. يشمل ذلك عدد التدفقات، وإمكانية عرض البيانات من الواجهة.
5. يجب أن يُسجل النظام جميع الأنشطة المهمة.
 - 1.5. تسجيل إضافة وحذف وتعديل قاعدة.
 - 2.5. تسجيل حالات حجب الاتصالات الشبكية الناتجة عن تطبيق قواعد جدار النار.
6. يجب أن يدعم النظام التَّحكُّم بالحركة اعتماداً على عنوان IP للمصدر (Source IP)، وعنوان IP للوجهة (Destination IP)، ونوع الميفاق (Protocol)، والمنفذ (Port).

4.1- المتطلبات غير الوظيفية

- الأداء (Performance): يجب أن يكون النظام سريع الاستجابة (يجب تطبيق القواعد بزمن استجابة شبه لحظي).
- الاعتمادية (Reliability): يجب أن يستمر النظام بالعمل أثناء إضافة القواعد أو حذفها.
- قابلية الاستخدام (Usability): يجب أن يكون للنظام واجهة وب بسيطة وواضحة، بحيث يمكن للمستخدم إضافة قاعدة دون معرفة تقنية عميقة.
- قابلية التوسع (Scalability): يجب أن يدعم النظام زيادة عدد القواعد مستقبلاً دون التأثير في أدائه.
- الأمن (Security): يجب أن يكون النظام محمي من إدخال البيانات الضارة.

وبذلك نكون قد شرحنا الهدف من المشروع واستعراض المتطلبات الوظيفية وغير الوظيفية الواجب على النظام أن يُجزّها. سنتعرف بالفصل القادم على المفاهيم النظرية التي استخدمناها في المشروع.

الفصل الثاني

الدراسة النظرية

يوضح هذا الفصل مجموعة من المفاهيم النظرية المستخدمة ضمن العمل.

1.2- الشبكات المعرفة برمجياً (Software-Defined Networks)

1.1.2- تعريف

تُعد الشبكات المعرفة برمجياً نموذجاً حديثاً في تصميم وإدارة الشبكات يهدف إلى زيادة مرونة الشبكة وتبسيط إدارتها. تعتمد هذه التقنية على مبدأ فصل مستوي التحكم عن مستوي تمرير البيانات، بحيث لا تكون المبدلات (Switches) مسؤولة عن اتخاذ قرارات التوجيه كما هو الحال في الشبكات التقليدية، وإنما تقتصر مهمتها على تمرير الرزم وفق قواعد يتم إرسالها من متحكم مركزي (Controller).

يمتلك المتحكم رؤية شاملة لحالة الشبكة، مما يتيح له إدارة حركة السير وتطبيق السياسات الأمنية وتعديلها بصورة مركزية وديناميكية، وهو ما يجعل الشبكة أكثر قابلية للإدارة والتطوير مقارنةً بالبنى التقليدية.

2.1.2- مزايا SDN

تقدم SDN مجموعة من المزايا:

- المرونة (Flexibility): إمكانية تعديل سياسات الشبكة دون الحاجة إلى إعادة تهيئة جميع الأجهزة.
- الإدارة المركزية (Centralized Management): التحكم في الشبكة بالكامل من خلال متحكم واحد.
- القابلية للبرمجة (Programmability): تطوير تطبيقات شبكية وأمنية جديدة بسهولة باستخدام واجهات برمجية.
- القابلية للتوسع (Scalability): إمكانية إدارة شبكات كبيرة بكفاءة عالية.
- سهولة الأتمتة (Automation): تنفيذ التغييرات وإدارة الشبكة بصورة آلية.

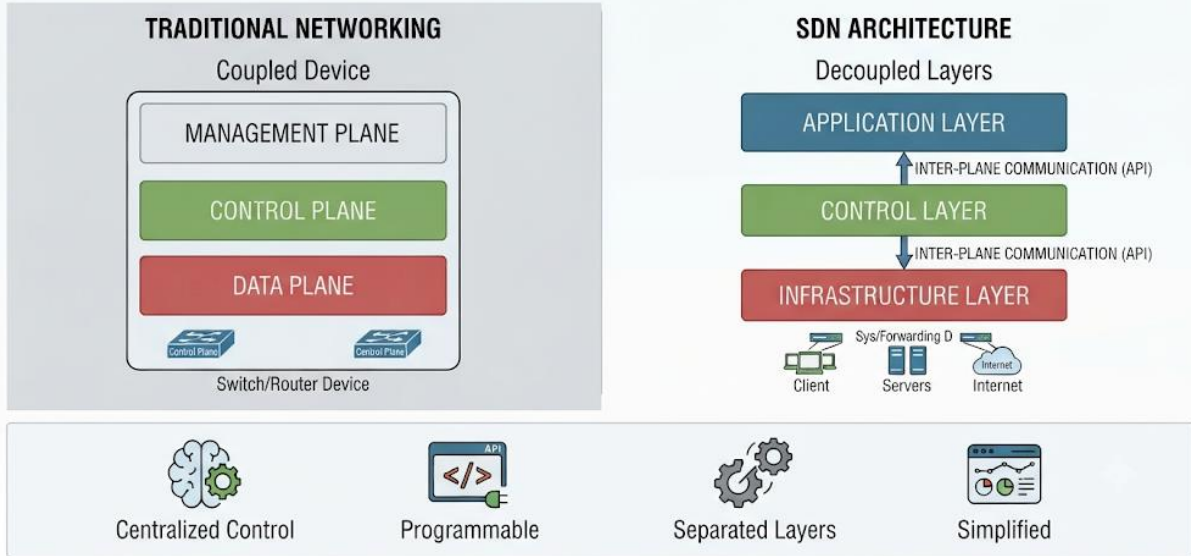
3.1.2- معمارية SDN

تتكون معمارية SDN من ثلاث طبقات رئيسية:

- **طبقة التطبيقات (Application Layer):** تضم التطبيقات التي توفر خدمات الشبكة مثل جدران النار وأنظمة كشف التسلل (Intrusion Detection Systems) وموازنة الأحمال.
 - **طبقة التحكم (Control Layer):** تتضمن المتحكم المركزي (مثل Ryu)، وهو المسؤول عن اتخاذ القرارات وإدارة الشبكة.
 - **طبقة البنية التحتية (Infrastructure Layer):** تضم المبدلات الداعمة لميفاق OpenFlow والتي تقوم بتنفيذ قواعد التوجيه فقط.
- يوضح المخطط التالي معمارية SDN.

SOFTWARE-DEFINED NETWORKING (SDN): DECOUPLED ARCHITECTURE

SIMPLIFIED ARCHITECTURAL VIEW



الشكل 1- معمارية الشبكات المعرفة برمجياً.

يُمثل الشكل 1 الفرق بين فصل الطبقات في شبكات SDN والشبكات التقليدية، حيث أصبح هناك تحكم مركزي في إدارة حركة السير ضمن الشبكة، بالإضافة إلى تطبيق السياسات الأمنية ديناميكياً.

2.2- ميفاق OpenFlow

1.2.2- تعريف

يُعد OpenFlow أول ميفاق قياسي يسمح بالتواصل بين متحكم SDN والمبدلات الشبكية (Open vSwitches (OVS. يحدد الميفاق الآلية التي يرسل بها المتحكم قواعد التوجيه إلى المبدلات ويستقبل منها معلومات حول الرزم المارة داخل الشبكة.

2.2.2- جداول التدفق (Flow Tables)

يعتمد OpenFlow على جداول تدفق داخل كل مبدلة شبكي، حيث تحتوي كل قاعدة على:

- شروط المطابقة (Match Fields)
- الإجراء (Action)
- الأولوية (Priority)
- العدادات (Counters)

عند وصول رزمة بيانات، يبحث المبدلة عن أول قاعدة تطابق خصائصها، ثم ينفذ الإجراء المحدد.

3.2.2- مفهومي Flow-Mod و Packet-In

إذا وصلت رزمة لا يوجد لها قاعدة مناسبة، يقوم المبدلة بإرسال رسالة Packet-In إلى المتحكم. عند عدم العثور على قاعدة مطابقة، يرسل المبدلة الرزمة إلى المتحكم، الذي يحللها ويحدد الإجراء المناسب، ثم يضيف قاعدة جديدة لمعالجة الرزم المماثلة مستقبلاً.

4.2.2- الأولويات في قواعد التدفق

قد تنطبق أكثر من قاعدة على الرزمة نفسها، لذلك يستخدم OpenFlow مفهوم الأولوية (Priority) لتحديد القاعدة التي سيتم تنفيذها.

القاعدة ذات الأولوية الأعلى تُنفذ أولاً، مما يسمح بإنشاء قواعد استثناء دقيقة داخل جدار النار.

3.2- جدار النار (Firewall)

1.3.2- تعريف

جدار النار هو نظام أمني يقوم بمراقبة حركة البيانات الداخلة والخارجة من الشبكة، ويحدد ما إذا كان سيتم السماح للزرم أو حجبتها وفق مجموعة من السياسات الأمنية المحددة مسبقاً.

2.3.2- أنواع جدران النار

- جدار نار من دون رقابة الحالة (Stateless Firewall): يفحص كل رزمة بصورة مستقلة دون معرفة حالة الاتصال.
- جدار نار برقابة الحالة (Stateful Firewall): يتابع حالة الاتصالات النشطة، وبالتالي يستطيع اتخاذ قرارات أكثر دقة.
- جدار نار طبقة التطبيق (Application Layer Firewall): يفحص البيانات على مستوى التطبيقات مثل HTTP و FTP، ويوفر مستوى حماية أعلى.

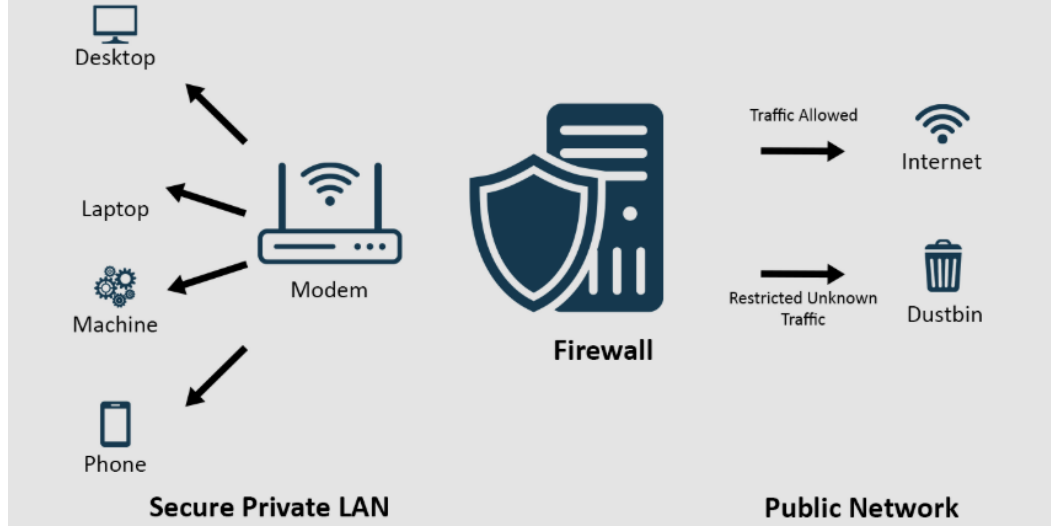
3.3.2- آلية السماح والحجب

يعتمد جدار النار في هذا المشروع على مقارنة الرزمة مع قواعد محددة تشمل:

- عنوان IP المصدر
- عنوان IP الوجهة
- المينافق
- المنفذ (عند الحاجة)

ثم يتخذ قرار سماح (Allow) أو منع (Block) وفقاً للقواعد المخزنة.

يُمثّل الشكل المرفق مبدأ عمل جدار النار.



الشكل 2- مبدأ عمل جدار النار.

يُظهر الشكل 2 تدفق حركة سير البيانات من أجهزة شبكة محلية خاصة (LAN) آمنة نحو شبكة عامة (الإنترنت). ويقوم جدار النار بدور الفلتر، حيث يسمح بمرور حركة السير المسموح بها ويمنع غير المعروفة والمقيدة.

4.2- متحكمات SDN

العنصر الرئيس في معمارية SDN، إذ تتولى إدارة الشبكة بصورة مركزية من خلال التواصل مع المبدلات باستخدام ميفاقات قياسية مثل OpenFlow. تمتلك هذه المتحكمات رؤية شاملة لحالة الشبكة، مما يتيح لها اتخاذ قرارات التوجيه، وتطبيق السياسات الأمنية، وإدارة تدفقات البيانات بصورة ديناميكية، بالإضافة إلى توفير واجهات برمجية تسمح للتطبيقات الخارجية بالتفاعل مع الشبكة وإدارتها. في هذا المشروع تم استخدام متحكم Ryu نظراً لدعمه ميفاق OpenFlow 1.3، ومرونته في تطوير تطبيقات SDN، إضافةً إلى توفيره واجهة REST API التي سهلت ربط واجهة الوب بالمتحكم وإدارة القواعد الأمنية.

5.2- REST API

REST هو أسلوب معماري لتطوير خدمات الوب يعتمد على الميفاق HTTP.

في المشروع استُخدمت الطريقتان:

- GET لاسترجاع قواعد جدار النار.
- POST لإضافة قاعدة جديدة.

يتم تبادل البيانات بين Flask و Ryu باستخدام JSON، وهو تنسيق خفيف وسهل القراءة.

6.2- ميفيق الشبكة المدعومة

- ميفاق Transmission Control Protocol (TCP): يعتمد على الاتصال ويضمن وصول البيانات بصورة صحيحة (رقمه 6).
- ميفاق User Datagram Protocol (UDP): ميفاق سريع لا يضمن إعادة إرسال البيانات (رقمه 17).
- ميفاق Internet Control Message Protocol (ICMP): يُستخدم في رسائل التشخيص مثل Ping (رقمه 1).

7.2- قواعد المعطيات (Databases)

تُعَدُّ قاعدة المعطيات أحد المكونات الأساسية في الأنظمة البرمجية، إذ تُستخدم لتنظيم المعطيات وتخزينها وإدارتها بصورة تضمن سهولة الوصول إليها واسترجاعها وتحديثها بكفاءة. تختلف نظم إدارة قواعد المعطيات باختلاف حجم التطبيقات ومتطلباتها، إلا أنها تشترك في توفير آليات للحفاظ على تكامل البيانات وموثوقيتها، ودعم تنفيذ عمليات الإضافة والتعديل والحذف والاستعلام بصورة آمنة ومنظمة.

8.2- تعلُّم الآلة والتعلُّم العميق (Machine Learning/Deep Learning)

تطوير خوارزميات ونماذج رياضية قادرة على التعلم من البيانات وتحسين أدائها تلقائيًا مع مرور الوقت، دون الحاجة إلى برمجتها بشكل صريح لكل حالة. يعتمد هذا المجال على تحليل البيانات واستخراج الأنماط والعلاقات بينها، بهدف بناء نماذج تستطيع إجراء التنبؤات أو التصنيفات أو اتخاذ القرارات استنادًا إلى الخبرة المكتسبة من بيانات التدريب. يوظف المشروع تقنيات تعلم الآلة لدعم تحليل البيانات الشبكية وتحسين دقة اكتشاف الأنشطة غير الطبيعية، بما يساهم في تعزيز كفاءة الجدار الناري القابل للبرمجة.

استعرض هذا الفصل المفاهيم النظرية التي يقوم عليها النظام المقترح، بما في ذلك الشبكات المعرفة برمجياً، وميفاق OpenFlow، وجدران النار. يمثل هذا الأساس النظري مرجعاً للفصل التالي، الذي يتناول الدراسات والأعمال السابقة ذات الصلة.

الفصل الثالث

الدراسة المرجعية

يتناول هذا الفصل الأبحاث المرتبطة بالعمل المقدم.

1.3- مقدمة

يهدف هذا الفصل إلى استعراض وتحليل الأدبيات البحثية والأعمال السابقة المتعلقة بتصميم وتطوير جدران النار القابلة للبرمجة باستخدام SDN، مع التركيز على الدراسات التي تناولت توظيف ميفاق OpenFlow ومتحكمات SDN في تطبيق السياسات الأمنية وإدارة حركة السير الشبكية. كما سيتم استعراض أبرز الأبحاث التي ناقشت مزايا جدران النار المبنية على SDN مقارنةً بجدران النار التقليدية، من حيث المرونة، والإدارة المركزية، وقابلية التوسع، والاستجابة الديناميكية للتهديدات الأمنية، وذلك لتحديد موقع هذا المشروع ضمن المشهد البحثي الحالي وتوضيح الإسهام الذي يقدمه في تصميم وتنفيذ جدار ناري قابل للبرمجة [1] [2] [3].

2.3- أداء متحكمات SDN

تناولت دراسة (Albu-Salih 2022) تقييم أداء متحكم Ryu في SDN من حيث معدل الإنتاجية والتأخير، وذلك باستخدام أداة Cbench ومحاكي Mininet [4]. أجرى الباحثون تقييماً شاملاً لأداء شبكة SDN باستخدام محاكي Mininet، وحلّلوا رسائل ميفاق OpenFlow باستخدام Wireshark، وقاسوا الإنتاجية والتأخير لمتحكم Ryu. توصلت الدراسة إلى أن Ryu يُعدّ خياراً مناسباً لبيئات البحث العلمي نظراً لمرونته وسهولة تطويره بلغة Python. في سياق مقارنة، أجرى باحثون دراسة لتقييم أداء متحكمي POX و Ryu في بيئات محاكاة متنوعة. استكشفت الدراسة أداء جودة الخدمة لمتحكمي SDN الشائعين POX و Ryu باستخدام Mininet للمحاكاة الشبكية [5]، حيث تفصل SDN مستويي التحكم والبيانات مما يتيح الإدارة المركزية وتحسين الأداء. أثبتت النتائج تفوّق Ryu في قابلية التوسع والتكيف مع البيئات الشبكية الأكثر تعقيداً، مما يدعم اختيارنا له كمتحكم في هذا المشروع.

3.3- جدران النار المبنية على SDN

قدّم الباحثون في دراسة نُشرت في (IEEE 2024) إطار عمل لجدار نار مبني على SDN مع تحديثات فورية للسياسات. تُقدّم الدراسة جدار نار مبني على SDN مع تحديثات فورية للسياسات، يدمج خوارزميات تعلّم الآلة للكشف التكيّفي عن التهديدات وتطبيق سياسات الأمان الديناميكية، من خلال المراقبة المستمرة لحركة مرور الشبكة وفحص الرزم [6]. تتقاطع هذه الدراسة مع مشروعنا في مفهوم التطبيق الديناميكي للقواعد الأمنية دون الحاجة إلى إعادة تشغيل المتحكم.

كما اقترح الباحثون في دراسة SDFW إطار عمل لجدار نار موزّع مبني على SDN. يستخدم جدار النار SDFW قواعد OpenFlow المطبّقة على المبدّلات ضمن إطار جدار نار موزّع، ويتوسع جيداً على الشبكات الكبيرة مع تأثير محدود على الأداء لا يتجاوز 1.6% في خفض عرض النطاق الترددي [7].

وفي دراسة مكّملة حول جدران النار ذات الحالة المبنية على SDN للحوسبة السحابية، اقترح الباحثون جدار نار ذا حالة مبنياً على مستوى البيانات القابل للبرمجة، قادراً على استخراج وتحليل وتسجيل معلومات حالة الاتصال للرزم في مستوى البيانات من خلال تصميم آلة حالة محدودة وجدول حالات [8].

4.3- التطبيق الديناميكي لقواعد التدفق

تناولت دراسة (Lin و Chang 2020) مسألة التوزيع الكفؤ لقواعد جدران النار الديناميكية في SDN. وأشارت الدراسة إلى أنه مع تزايد ديناميكية البيئات الشبكية وتعقيدها، يمكن استخدام جدران النار الموزعة مع لوائح التحكم في النفاذ (Access Control Lists) لحماية الخدمات والمضيفين الداخليين من الهجمات الشبكية والهجمات الداخلية. واقترحت الدراسة خوارزمية تقسيم مقيدة بالموارد (Resource Constraint Splitting Algorithm) لحساب القواعد المرتبطة بالسياسات التي تم تحديثها فقط، مع الحفاظ على القواعد غير المتأثرة، مما يؤدي إلى تقليل زمن الحساب والعبء الناتج عن تحديث مواضع القواعد [9].

كما اقترح Hu وآخرون إطار عمل شاملاً يُسمى FlowGuard لتسهيل الكشف الدقيق عن انتهاكات سياسات جدار النار ومعالجتها في شبكات OpenFlow الديناميكية. يعتمد الإطار على فحص مساحات تدفق الشبكة للكشف عن انتهاكات السياسات عند حدوث تغيرات في حالة الشبكة، كما يوفر آليات آلية وفورية لمعالجة هذه الانتهاكات. أظهرت التجارب التي أجريت باستخدام طوبولوجيا شبكية واقعية فعالية الإطار وكفاءة مقبولة في اكتشاف الانتهاكات ومعالجتها [10].

5.3- موقع المشروع الحالي ومساهمته المقترحة

على الرغم من الجهود البحثية المبذولة في مجال جدران النار المبنية على SDN، إلا أن استعراض الدراسات السابقة يكشف عن جملة من الفجوات:

- غياب واجهة المستخدم التفاعلية: من خلال نطاق الدراسات التي تمت مراجعتها، لوحظ أن التركيز الرئيس انصب على الجانب النظري وخوارزميات توزيع القواعد، دون تقديم واجهة وب عملية تُمكن مدير النظام من إدارة السياسات الأمنية بسهولة.
- ضعف الاستمرارية: أغلب الأنظمة المقترحة تخزن القواعد في ذاكرة المتحكم مؤقتاً، مما يعني فقدان جميع السياسات عند إعادة تشغيل المتحكم.
- محدودية بيئة الاختبار: تعتمد معظم الدراسات على Mininet التقليدي الذي يُنشئ مضيفين افتراضيين بسيطين، دون محاكاة بيئات واقعية تحتوي على خدمات حقيقية.
- غياب نظام السجلات المتكامل: لا تتضمن أغلب الأنظمة المقترحة آلية شاملة لتسجيل الأحداث الأمنية وتتبعها بشكل دائم.
- نظام الأولويات: تفتقر كثير من الأنظمة إلى آلية صريحة وموثقة للتعامل مع تعارض القواعد.

تتمثل المساهمة الرئيسية المقترحة لهذا المشروع في تقديم تصميم وتنفيذ متكامل لجدار نار مبني على SDN يجمع بين:

1. التكامل الكامل بين مستويات النظام (متحكم Ryu + واجهة Flask مع REST API + محاكاة Containernet).

2. واجهة وب تفاعلية واحترافية.

3. تخزين القواعد والسجلات بشكل دائم.

4. نظام أولويات صريح لحل تعارض القواعد.

وهكذا نكون قد استعرضنا أهم الدراسات المرجعية المستند عليها في هذا المشروع. ننتقل في الفصل القادم إلى دراسة تحليلية للنظام.

الفصل الرابع

الدراسة التحليلية

يقدم هذا الفصل عملية تحليل النظام ودراسة متطلباته.

1.4- تحديد الفاعلين (Actors)

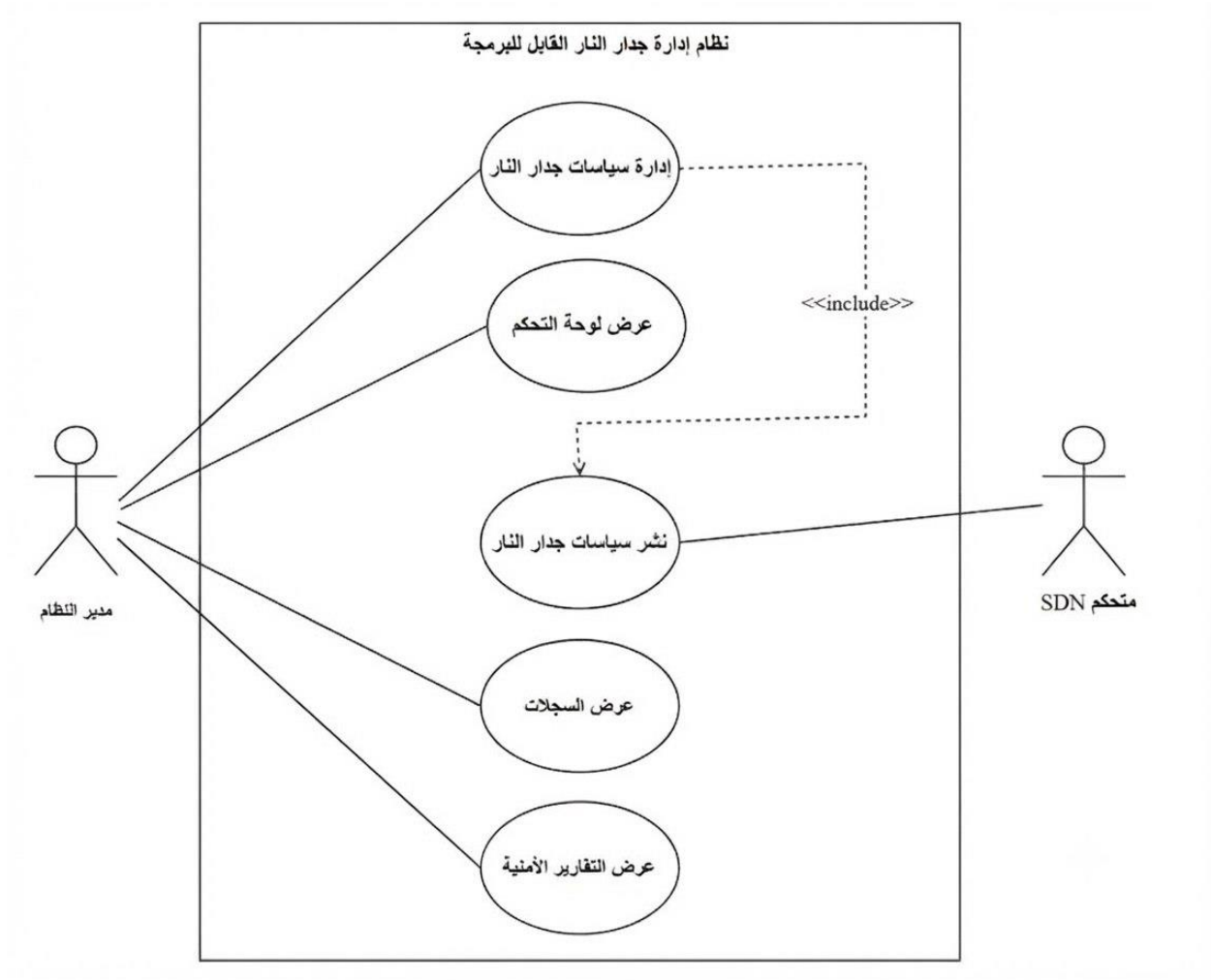
– مدير النظام (System Administrator)

الفاعل الرئيسي في النظام، وهو المسؤول عن إدارة سياسات جدار النار من خلال إضافة القواعد وتعديلها وحذفها، بالإضافة إلى استعراض لوحة التحكم، ومتابعة السجلات، والاطلاع على التقارير الأمنية لمراقبة أداء النظام واتخاذ الإجراءات المناسبة.

– متحكم الشبكة المعرفة برمجياً (SDN Controller)

يمثل المكون المسؤول عن تنفيذ سياسات جدار النار داخل بيئة الشبكة المعرفة برمجياً. يستقبل السياسات المرسله من النظام ويحوّلها إلى قواعد OpenFlow، ثم يقوم بتوزيعها على مبدلات OVS لتطبيقها على حركة السير الشبكية.

2.4- مخطط حالات الاستخدام (Use Case Diagram)



الشكل 3- مخطط حالات الاستخدام.

يبين الشكل 3 مخطط حالات الاستخدام التفاعل بين الفاعلين الرئيسيين ووظائف النظام الأساسية، حيث يتولى مدير النظام إدارة سياسات جدار النار واستعراض بيانات النظام، بينما يتولى متحكم SDN تطبيق السياسات على الشبكة لضمان تنفيذها.

3.4- السرد النصي لحالات الاستخدام

1.3.4- إدارة سياسات جدار النار

الجدول 1- إدارة سياسات جدار النار.

اسم الحالة: إدارة سياسات جدار النار	
الوصف (Description)	تمكّن هذه الحالة مدير النظام من إدارة سياسات جدار النار من خلال إضافة السياسات أو تعديلها أو حذفها، ليقوم النظام بعد ذلك بتطبيق التعديلات على بيئة الشبكة المعرفة برمجياً باستخدام متحكم SDN.
الفاعلون (Actors)	مدير النظام.
الشروط السابقة (Preconditions)	- تشغيل النظام بصورة صحيحة. - اتصال متحكم Ryu بالشبكة. - جاهزية قاعدة المعطيات لحفظ السياسات.
الشروط اللاحقة (Postconditions)	- تم تحديث قاعدة معطيات السياسات. - تم تطبيق السياسات الجديدة على مبدّلات OpenFlow. - أصبحت السياسات الجديدة فعالة على حركة السير الشبكية.

سير الأحداث

السيناريو الأساسي الناجح

الجدول 2- السيناريو الأساسي الناجح لحالة استخدام إدارة سياسات جدار النار.

النظام	مدير النظام
	1. يختار صفحة إدارة السياسات.
	2. يقوم بإضافة سياسة جديدة أو تعديل سياسة موجودة أو حذفها.
3. يتحقق النظام من صحة بيانات السياسة المدخلة.	
4. يحفظ التعديلات في قاعدة المعطيات.	
5. يحول السياسات إلى قواعد OpenFlow.	
6. يرسل القواعد إلى متحكم Ryu.	

7. يقوم متحكم Ryu بتطبيق القواعد على مبدلات OVS.	
8. يعرض النظام رسالة تؤكد نجاح العملية.	

المسارات البديلة

A1: يقوم مدير النظام بإلغاء العملية قبل الحفظ.

- يقوم النظام بإلغاء التعديلات .
- تبقى السياسات الحالية دون تغيير.

مسارات الأخطاء

E1: في الخطوة (3)، تحتوي السياسة على بيانات غير صحيحة.

- يعرض النظام رسالة توضح سبب الخطأ.
- يطلب من مدير النظام تصحيح البيانات.

E2: في الخطوة (4)، يفشل حفظ السياسة في قاعدة المعطيات.

- يسجل النظام الخطأ.
- يعرض رسالة تفيد بعدم نجاح العملية.

E3: في الخطوة (6) أو (7)، يتعذر الاتصال بمتحكم Ryu أو يفشل تطبيق القواعد.

- يسجل النظام الخطأ.
- يحتفظ بالسياسات في قاعدة المعطيات دون تطبيقها على الشبكة.
- يعرض رسالة تنبيه لمدير النظام.

2.3.4- عرض لوحة التحكم

الجدول 3- عرض لوحة التحكم.

اسم الحالة: عرض لوحة التحكم	
الوصف (Description)	تتيح هذه الحالة لمدير النظام عرض لوحة التحكم التي تتضمن ملخصاً عن حالة النظام، مثل عدد سياسات جدار النار، وعدد الأحداث الأمنية، وبعض الإحصائيات العامة المتعلقة بالنظام.
الفاعلون (Actors)	مدير النظام.

الشروط السابقة (Preconditions)	- تشغيل النظام بصورة صحيحة. - توفر البيانات اللازمة لعرض الإحصائيات.
الشروط اللاحقة (Postconditions)	- تم عرض لوحة التحكم بنجاح متضمنة أحدث المعلومات المتوفرة.

سير الأحداث

السيناريو الأساسي الناجح

الجدول 4- السيناريو الأساسي الناجح لحالة استخدام عرض لوحة التحكم.

النظام	مدير النظام
	1. يختار صفحة لوحة التحكم.
2. يجلب النظام البيانات والإحصائيات المطلوبة.	
3. يعالج البيانات ويعرضها ضمن لوحة التحكم.	

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: فشل النظام في استرجاع البيانات.

- يعرض النظام رسالة تفيد بعدم إمكانية تحميل لوحة التحكم.
- يسجل الخطأ في سجل النظام.

3.3.4- عرض التقارير الأمنية

الجدول 5- عرض التقارير الأمنية.

اسم الحالة: عرض التقارير الأمنية	
الوصف (Description)	تتيح هذه الحالة لمدير النظام الاطلاع على التقارير الأمنية التي تعرض ملخصاً للأحداث الأمنية المكتشفة، مثل عدد الاتصالات المحظورة والأنشطة المشبوهة والإحصائيات المتعلقة بحركة السير الشبكية.
الفاعلون (Actors)	مدير النظام.
الشروط السابقة (Preconditions)	- تشغيل النظام بصورة صحيحة. - توفر بيانات التقارير الأمنية.
الشروط اللاحقة (Postconditions)	- تم عرض التقارير الأمنية بنجاح.

سير الأحداث

السيناريو الأساسي الناجح

الجدول 6- السيناريو الأساسي الناجح لحالة استخدام عرض التقارير الأمنية.

النظام	مدير النظام
	1. يختار صفحة التقارير الأمنية.
2. يسترجع النظام بيانات التقارير الأمنية من قاعدة المعطيات.	
3. يعرض النظام التقارير والإحصائيات الأمنية.	

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: تعذر استرجاع بيانات التقارير.

- يعرض النظام رسالة توضح فشل تحميل التقارير.
- يسجل الخطأ في سجل النظام.

4.3.4- عرض السجلات

الجدول 7- عرض السجلات.

اسم الحالة: عرض السجلات	
الوصف (Description)	تتيح هذه الحالة لمدير النظام استعراض السجلات التي يوثقها النظام، والتي تتضمن الأحداث المتعلقة بسياسات جدار النار والعمليات المنفذة داخل النظام.
الفاعلون (Actors)	مدير النظام.
الشروط السابقة (Preconditions)	- تشغيل النظام بصورة صحيحة. - وجود سجلات محفوظة في قاعدة المعطيات.
الشروط اللاحقة (Postconditions)	- تم عرض السجلات المطلوبة بنجاح.

سير الأحداث

السيناريو الأساسي الناجح

الجدول 8- السيناريو الأساسي الناجح لحالة استخدام عرض السجلات.

النظام	مدير النظام
	1. يختار صفحة السجلات.
2. يسترجع النظام السجلات من قاعدة المعطيات.	
3. يعرض النظام السجلات مرتبة حسب وقت تسجيلها.	

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: فشل استرجاع السجلات من قاعدة المعطيات.

- يعرض النظام رسالة توضح سبب الفشل.
- يسجل الخطأ في سجل النظام.

الفصل الخامس

المنهجية المقترحة

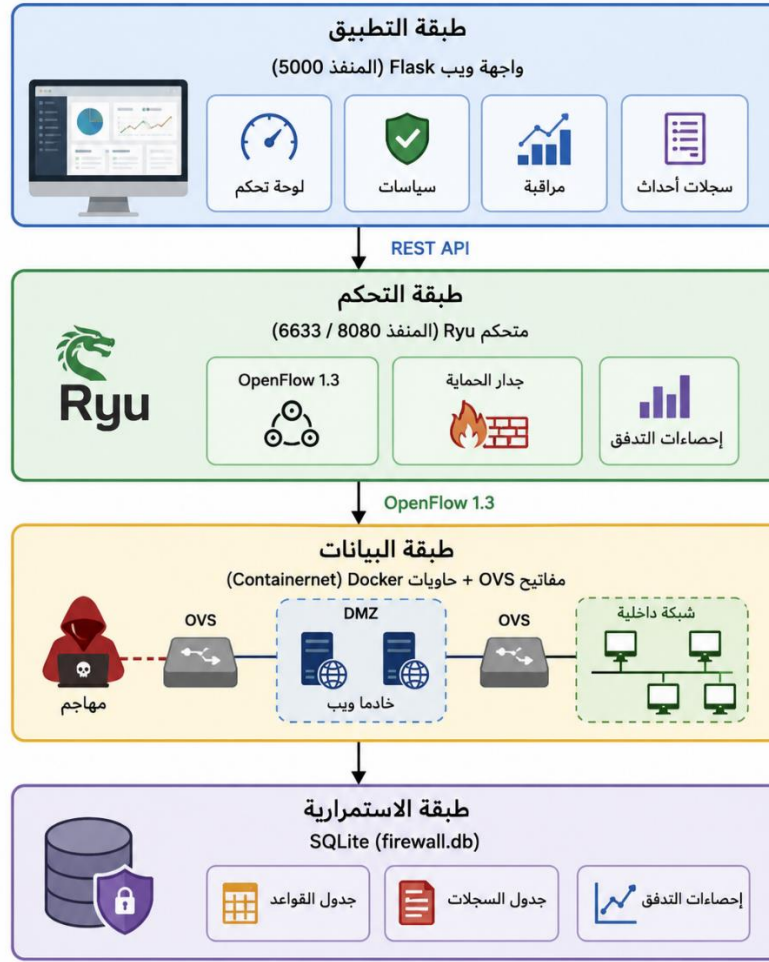
يُقدّم هذا الفصل منهجية العمل المقترحة لتحقيق النظام.

1.5- مقدمة

بناءً على الأهداف المحددة في الفصول السابقة والتحديات المتعلقة بإدارة الأمن الشبكي في الشبكات التقليدية، يهدف هذا الفصل إلى تقديم المنهجية المقترحة لتصميم نظام جدار النار القابل للبرمجة وتنفيذه. تستند هذه المنهجية إلى دمج مكونين رئيسيين: مستوى التحكم المركزي المنفذ عبر متحكم Ryu باستخدام ميفاق OpenFlow 1.3، ومستوى التطبيق المنفذ عبر واجهة وب تفاعلية مبنية بإطار عمل Flask. يهدف هذا الدمج إلى تحقيق التوازن بين الكفاءة التشغيلية والمرونة في إدارة السياسات الأمنية. سيتم الاعتماد على مجموعة من التقنيات المتخصصة في معالجة حركة السير الشبكية، حيث يستخدم متحكم Ryu ميفاق OpenFlow 1.3 للتحكم المركزي في المبدلات الافتراضية OVS، كما يستخدم النظام قاعدة معطيات SQLite لضمان استمرارية السياسات الأمنية وسجلات الأحداث عبر دورات إعادة التشغيل.

2.5- النموذج المقترح

تعتمد المنهجية المقترحة على معمارية رباعية الطبقات تعمل بشكل متكامل، حيث تضطلع كل طبقة بمهام محددة وتتكامل مع الطبقات الأخرى لتوفير نظام حماية شبكية متكامل وفعال. يوضح الشكل 4 المعمارية المقترحة للنظام.



الشكل 4- معمارية النظام المقترح.

يوضح الشكل 4 البنية المعمارية المقترحة للنظام، حيث تتكون من أربع طبقات مترابطة تبدأ بطبقة التطبيق التي توفر واجهة إدارة عبر Flask، ثم طبقة التحكم المعتمدة على متحكم Ryu، والتي تتواصل مع طبقة البيانات التي تضم مبدلات OVS، بينما تتولى طبقة الاستمرارية حفظ قواعد جدار النار، وسجلات الأحداث، وإحصائيات التدفق في قاعدة معطيات SQLite لضمان استمرارية عمل النظام وتتبع نشاطه.

1.2.5- منهجية طبقة التحكم (Controller Layer)

الهدف الأساسي: تهدف إلى توفير واجهة مركزية للمسؤول لإدارة سياسات الجدار الناري ومراقبة حالة الشبكة وعرض السجلات والإحصائيات بطريقة سهلة وتفاعلية.

1.1.2.5- استقبال اتصالات المبدلات وتثبيتها

- الهدف: تهيئة المبدلات وتطبيق السياسات الأمنية المحفوظة تلقائياً.
- مصدر الأحداث: اتصال مبدلات OVS الافتراضية بالمتحكم عبر ميفاق OpenFlow 1.3 على المنفذ 6633.
- الخطوات المنهجية:
 - استقبال حدث الاتصال: عند إنشاء اتصال بين مبدلة الشبكة والمتحكم، يتم تهيئة عملية التواصل بينهما وفق ميفاق OpenFlow، مما يسمح للمتحكم بإدارة حركة السير داخل الشبكة.
 - تثبيت القاعدة الافتراضية: يُثبت قاعدة تدفق ذات أولوية صفر لإرسال جميع الرزم غير المطابقة إلى المتحكم لمعالجتها.
 - إعادة تطبيق القواعد المحفوظة: تُحمّل جميع القواعد من قاعدة المعطيات وتُطبّق على المبدلة الجديد لضمان الاستمرارية.
- الخرج: مبدلات OVS جاهزة لتطبيق السياسات الأمنية فور الاتصال.

2.1.2.5- معالجة رزم المرور

- الهدف: فحص كل رزمة واردة وتطبيق قواعد الحجب أو السماح وفق السياسات المعروفة.
- الخطوات المنهجية:
 - استخراج معلومات الرزمة: استخراج عنوان IP المصدر والوجهة والميفاق والمنفذ.
 - مطابقة قواعد الحجب: مقارنة معلومات الرزمة مع قواعد الحجب المفعلة بترتيب الأولوية.
 - تطبيق الإجراء المناسب:
 - في حالة التطابق مع قاعدة حجب: تجاهل الرزمة وتسجيل الحدث في قاعدة المعطيات.
 - في حالة عدم التطابق: توجيه الرزمة إلى وجهتها وتثبيت قاعدة تدفق للرزم المتبعة.
- الخرج: رزم المرور مفلترة وفق السياسات الأمنية المعروفة مع تسجيل جميع الأحداث.

3.1.2.5- مراقبة إحصائيات التدفق

- الهدف: رصد الرزم المحجوبة بشكل دقيق بعد تثبيت قواعد Drop Flow على المبدلات.

– الخطوات المنهجية:

- إطلاق نيسب (Thread) المراقبة: عند بدء تشغيل المتحكم، يُنشأ نيسب مستقل لاستطلاع إحصائيات التدفق.
- إرسال طلبات الإحصائيات: يقوم المتحكم بشكل دوري بجمع إحصائيات حركة السير من جميع مبدلات الشبكة المتصلة لمتابعة حالة التدفقات.
- معالجة الردود: تُقارن الإحصائيات المستلمة مع القيم المسجلة سابقاً للكشف عن أي زيادة في عدد الرزم التي تمت معالجتها بواسطة قواعد الحجب.
- تسجيل الأحداث الجديدة: عند اكتشاف زيادة في عدد الرزم المحجوبة، يُسجل حدث أمني جديد في قاعدة المعطيات لتوثيق عملية الحجب ومتابعتها.

– الخرج: سجل دقيق لجميع رزم المرور المحجوبة مع تحديث تلقائي كل 5 ثواني.

2.2.5- منهجية طبقة التطبيق (Application Layer)

الهدف الأساسي: تهدف طبقة التطبيق إلى توفير واجهة مركزية للمسؤول لإدارة سياسات جدار النار ومراقبة حالة الشبكة وعرض السجلات والإحصائيات بطريقة سهلة وتفاعلية.

1.2.2.5- إدارة السياسات الأمنية

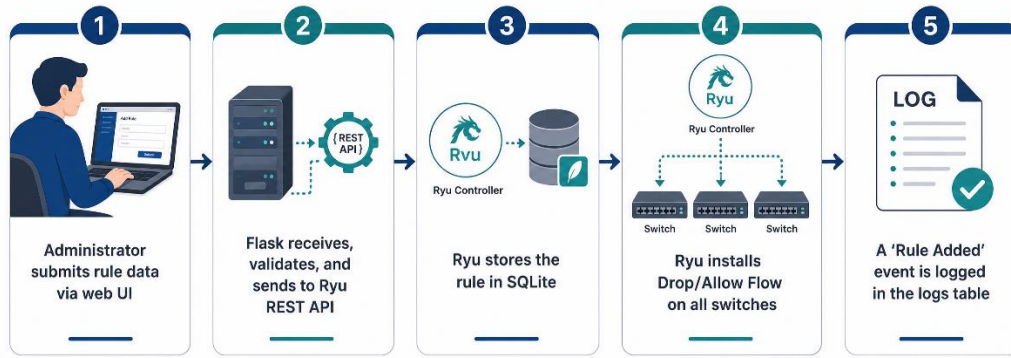
– الهدف: توفير واجهة وب تفاعلية تُمكن مدير النظام من إدارة قواعد جدار النار بسهولة ودون الحاجة إلى سطر الأوامر.

– الخطوات المنهجية:

- إضافة قاعدة جديدة: يُحدّد مدير النظام عنوان IP المصدر والوجهة والميفاق والمنافذ والإجراء والأولوية عبر نموذج وب تفاعلي.
- التحقق من صحة المدخلات: يُتحقق من صحة عناوين IP ونطاق المنافذ وعدم تطابق المصدر والوجهة قبل الإرسال.
- إرسال الطلب إلى المتحكم. يُرسل Flask طلب POST إلى Ryu.
- تطبيق القاعدة فوراً. يُثبت المتحكم قاعدة التدفق المناسبة على جميع المبدلات المتصلة ويحفظها في قاعدة المعطيات.

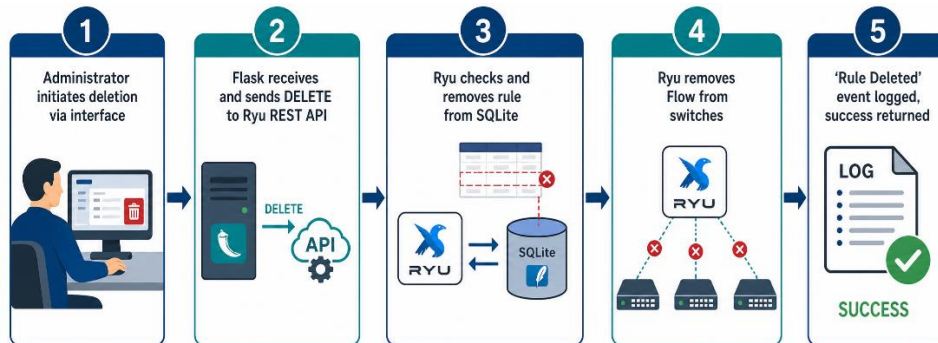
- **حذف قاعدة موجودة:** يختار مدير النظام القاعدة المراد حذفها من قائمة السياسات، ثم يرسل النظام طلب الحذف إلى متحكم Ryu لإزالة قاعدة التدفق (Flow Rule) من جميع مبدلات OVS، وبعد نجاح العملية تُحذف القاعدة من قاعدة المعطيات ويُسجّل الحدث في سجل النظام.
 - **التعديل على قاعدة موجودة:** يختار مدير النظام القاعدة المراد تعديلها ويُجري التغييرات المطلوبة على خصائصها، ثم يتحقق النظام من صحة البيانات، ويُحدّث القاعدة في قاعدة المعطيات، ويُعيد تطبيقها على مبدلات OVS، مع تسجيل عملية التعديل في سجل النظام.
- الخرج: قاعدة أمنية جديدة مُطبّقة فوراً على الشبكة ومحفوظة بشكل دائم (أو نجاح حذف قاعدة في حالة الحذف، أو قاعدة جديدة نتجت من التعديل على قاعدة موجودة في حالة التعديل).

تُمثّل الأشكال التالية سير عمليات إضافة وحذف والتعديل على قاعدة موجودة.



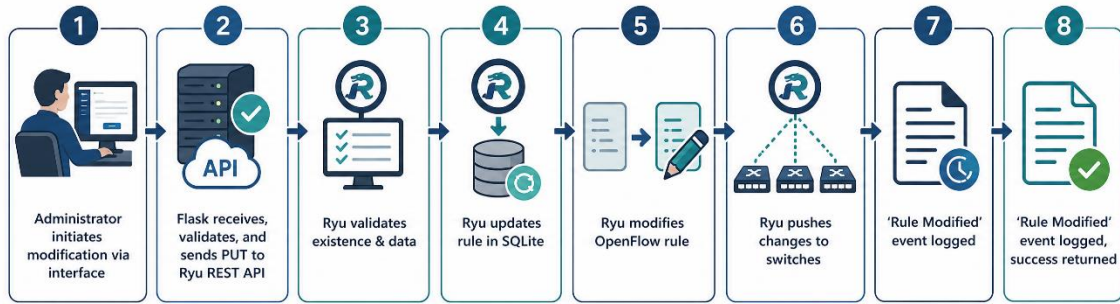
الشكل 5- سير عملية إضافة قاعدة وتطبيقها في النظام.

يوضح الشكل 5 تسلسل خطوات إضافة قاعدة جديدة إلى النظام، بدءاً من إدخالها عبر واجهة مدير النظام وحتى تخزينها وتطبيقها على جميع مبدلات الشبكة.



الشكل 6- سير عملية حذف قاعدة موجودة.

يوضح الشكل 6 مراحل معالجة طلب الحذف بدءاً من إرساله عبر الواجهة، مروراً بإزالة القاعدة من قاعدة المعطيات ومبدلات الشبكة، وانتهاءً بتسجيل الحدث وإرجاع رسالة النجاح.



الشكل 7- سير عملية التعديل على قاعدة موجودة.

يوضح الشكل 7 تسلسل خطوات تحديث السياسة بدءاً من التحقق من صحة البيانات ووجودها، ثم تحديثها في قاعدة المعطيات ومبدلات OVS، وصولاً لتسجيل التعديل بنجاح.

2.2.2.5- مراقبة حركة السير وعرض التقارير

- الهدف: توفير رؤية شاملة وفورية لحالة الشبكة والأحداث الأمنية عبر لوحة تحكم تفاعلية.
- الخطوات المنهجية:
 - الاستطلاع التلقائي: تجري الواجهة عملية تحديث دورية لعرض أحدث المعلومات المتعلقة بالقواعد وسجلات الأحداث.
 - عرض الإحصائيات: تعرض لوحة التحكم عدد القواعد النشطة وعدد الرزم المحجوبة وحالة المتحكم وعدد إجمالي الأحداث.
 - الرسوم البيانية: تُعرض رسوم بيانية خطية تُوضّح التغيّر التراكمي لأحداث الحجب وتعديلات القواعد عبر الزمن.
 - التوزيع حسب الميفاق: تُعرض نسب توزيع القواعد حسب الميفاق (TCP/UDP/ICMP) بأشرطة تقدم بصرية.
 - جدول تقرير الحجب: يعرض تفاصيل كل رزمة محجوبة تشمل الطابع الزمني وعناوين IP المصدر والوجهة والميفاق والمنفذ.
- الخرج: لوحة مراقبة شاملة تُحدّث تلقائياً وتوفّر رؤية كاملة للأنشطة الأمنية.

3.5- آلية التكامل بين الطبقات ومنطق تطبيق القواعد

يعتمد نجاح النظام على التكامل الفعّال بين طبقة التحكم وطبقة التطبيق، بالإضافة إلى كيفية تطبيق القواعد الأمنية على حركة السير.

- تدفق البيانات من طبقة التطبيق: تقوم واجهة الوب بإرسال طلبات REST API إلى متحكم Ryu، والذي يتولى تخزين القواعد في قاعدة المعطيات وتطبيقها فوراً على المبدّلات.
- الاستجابة الفورية للأحداث الجديدة: يعالج المتحكم الرزم الواردة فور وصولها، ويطبق سياسات الشبكة المناسبة لضمان توجيهها أو حجبتها وفقاً للقواعد الأمنية المعتمدة.
- منطق خدمة القواعد: عند تلّقي طلب تعديل قاعدة يتم:
 - حذف قاعدة التدفق القديمة من جميع المبدّلات.
 - تثبيت قاعدة التدفق الجديدة على جميع المبدّلات.
 - تحديث قاعدة المعطيات بالقاعدة الجديدة.
 - تسجيل حدث "Rule Modified" في سجل الأحداث.

قدّم هذا الفصل المنهجية الشاملة المقترحة لتصميم النظام، موضّحاً كيفية تكامل مكوناته المختلفة ضمن منظومة متماسكة تحقق المتطلبات الوظيفية المحددة. يتناول الفصل القادم التصميم التفصيلي للنظام، مع توضيح القرارات التصميمية المعتمدة وآلية تنظيم مكوناته.

الفصل السادس

تصميم النظام

يُفصّل هذا الفصل القرارات التصميمية التي بني من خلالها النظام.

1.6- مقدمة

بعد الانتهاء من تحليل متطلبات النظام وتحديد المنهجية المقترحة لتحقيقه، تأتي مرحلة تصميم النظام التي تمثل الجسر بين التحليل والتنفيذ. يهدف هذا الفصل إلى توضيح التصميم المعماري للنظام، وآلية ترابط مكوناته، وتصميم قاعدة المعطيات، بالإضافة إلى تصميم الوحدات البرمجية وواجهات المستخدم. وقد روعي في التصميم تحقيق البساطة وسهولة الصيانة وقابلية التوسع.

2.6- تصميم مكونات النظام

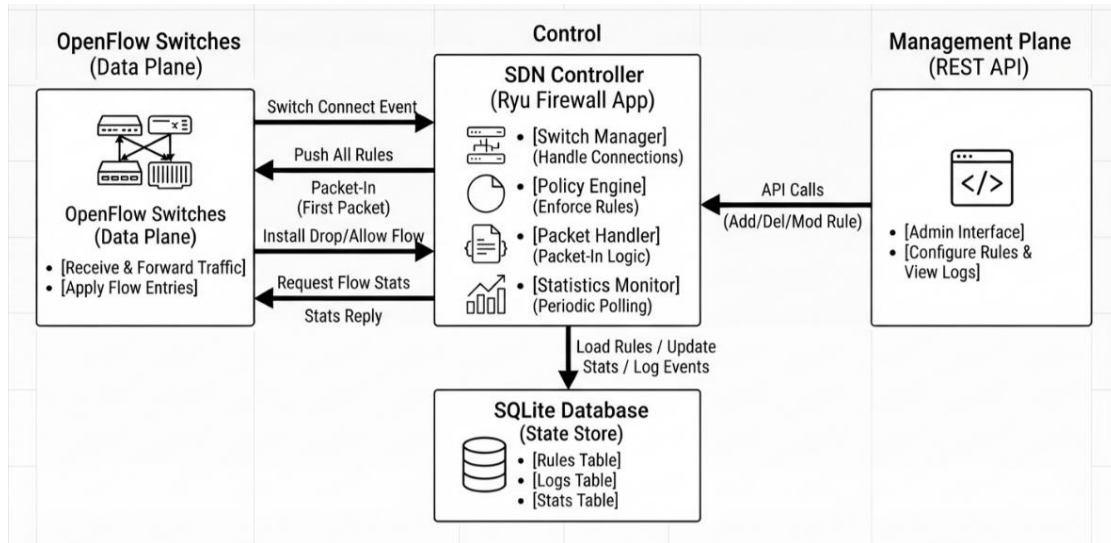
اعتمد النظام في تصميمه على معمارية التطبيق الأحادي (Monolithic Application Architecture)، بحيث تتكامل مكونات النظام البرمجية ضمن تطبيق واحد مترابط بدلاً من توزيعها على مجموعة من الخدمات المستقلة. تم اختيار هذه المعمارية بما يتناسب مع طبيعة المشروع وحجمه، إذ تتيح تبسيط عملية التطوير والإدارة، وتقليل التعقيد الناتج عن الاتصال بين الخدمات، مع الحفاظ على الفصل المنطقي بين نسيقات النظام ووظائفها المختلفة.

1.2.6- نسيقة المتحكم (Controller Module)

تُعد نسيقة المتحكم المكون الأساسي في النظام، حيث تمثل مستوي التحكم في بنية SDN، وتتولى الإدارة المركزية لجميع مبدلات OVS المتصلة بالشبكة. تعتمد هذه النسيقة على متحكم Ryu المطور بلغة Python، والذي يدعم ميفاق OpenFlow 1.3، مما يتيح التحكم المباشر في قواعد التدفق وإدارة حركة السير داخل الشبكة ديناميكياً. تعمل نسيقة المتحكم كوسيط بين واجهة إدارة النظام ومستوي تقرير البيانات، إذ تستقبل سياسات جدار النار من خدمة الواجهة عبر واجهة REST API، ثم تقوم بتحويلها إلى قواعد تدفق (Flow Rules) متوافقة مع ميفاق OpenFlow، وإرسالها إلى مبدلات OVS لتطبيقها على حركة السير. كما تستقبل الأحداث الواردة من المبدلات، مثل الرزم التي لا تطابق أي قاعدة تدفق، وتتخذ الإجراءات المناسبة لمعالجتها.

– المسؤوليات الرئيسية:

- استقبال اتصالات مبدلات OVS عبر المنفذ 6633 باستخدام ميفاق OpenFlow 1.3.
 - إدارة دورة حياة قواعد التدفق، بما يشمل إنشاء القواعد الجديدة، وتعديلها، وحذفها وفقاً للتغييرات التي يجريها مدير النظام.
 - معالجة الرزم الواردة التي لا تطابق أيّاً من قواعد التدفق، واتخاذ القرار المناسب بشأنها وفقاً لسياسات جدار النار.
 - تشغيل نيسب مستقل لمراقبة حالة الشبكة بصورة دورية، حيث يتم استطلاع إحصائيات التدفق وإحصائيات المنافذ كل خمس ثوانٍ، مما يتيح متابعة أداء الشبكة واستخراج البيانات اللازمة للتقارير الأمنية.
 - توفير واجهة REST API تعمل على المنفذ 8080 لاستقبال طلبات تطبيق الويب، مثل إضافة السياسات أو تعديلها أو حذفها، والاستجابة لهذه الطلبات بعد تنفيذها.
 - الحفاظ على اتصال مستمر مع جميع المبدلات الافتراضية المتصلة، وضمان تزامن قواعد التدفق بينها وبين السياسات المخزنة داخل النظام.
- كما تُسهم في تحقيق الإدارة المركزية للشبكة، إذ تُمكن مدير النظام من التحكم بجميع مبدلات OVS من نقطة واحدة دون الحاجة إلى تهيئة كل مبدلة بصورة منفصلة، الأمر الذي يزيد من مرونة إدارة الشبكة، ويقلل زمن تطبيق السياسات، ويُسهّل تحديثها بما يتوافق مع متطلبات الأمن الشبكي.
- يمثل المخطط التالي التفاعل بين متحكم SDN ومبدلات OpenFlow.



الشكل 8- آلية تفاعل مبدلات OpenFlow مع متحكم Ryu.

يوضح الشكل 8 هيكلية متحكم جدار النار المبرمج عبر Ryu، مبيناً تفاعل مبدلات OpenFlow مع محرك السياسات وإحصائيات التدفق. كما يظهر تكامل قاعدة معطيات SQLite لتخزين الحالة وواجهة REST API لإدارة القواعد وسجلات الأحداث بكفاءة.

2.2.6- نسيقة إدارة القواعد (Rules Management Module)

تُعد نسيقة إدارة القواعد مسؤولة عن إدارة دورة حياة سياسات جدار النار داخل النظام، بدءاً من إنشاء القاعدة والتحقق من صحة بياناتها، مروراً بتخزينها في قاعدة المعطيات، وانتهاءً بتطبيقها على مبدلات الشبكة وتسجيل العمليات المرتبطة بها. يمثل هذا المكون حلقة الوصل بين واجهة الإدارة ومتحكم الشبكة، حيث تضمن أن تكون جميع السياسات المخزنة متزامنة مع القواعد المطبقة فعلياً داخل شبكة SDN.

– المسؤوليات الرئيسية:

- استقبال طلبات إدارة السياسات الواردة من واجهة الوب عبر REST API.
- التحقق من صحة البيانات المدخلة قبل تنفيذ أي عملية.
- تخزين السياسات في جدول rules داخل قاعدة المعطيات.
- إنشاء سجل إحصائي لكل قاعدة في جدول rules_stats لمتابعة نشاطها.
- استدعاء خدمة المتحكم لتطبيق السياسة الأمنية على الشبكة بعد التحقق من صحتها وتخزينها.
- تسجيل جميع العمليات المنفذة في جدول logs لتوفير سجل تدقيق (Audit Trail).
- إعادة نتيجة العملية إلى واجهة المستخدم.

– نظام الأولويات: اعتمد نظام أولويات صريح لضمان تطبيق السياسات الأمنية بشكل صحيح عند تعارض القواعد:

الجدول 9- نظام الأولويات في القواعد.

نوع القاعدة	الأولوية	الوصف
قواعد الحجب	200	الأعلى أولوية
قواعد السماح	100	أولوية متوسطة
القاعدة الافتراضية	0	إرسال الرزم للمتحكم

يوضح الجدول 9 أولوية كل قاعدة في النظام، حيث تحتل قواعد الحجب الأولوية الأعلى مقارنة بقواعد السماح، أي تغلب قاعدة الحجب عند تعارض القواعد، مما يضمن عدم السماح بحركة سير تم تحديدها صراحةً على أنها غير آمنة، بينما تأخذ

القواعد الافتراضية (تلك التي يتم تطبيقها على المتحكم قبل إضافة أي قاعدة من الواجهة) الأولوية 0، لضمان تنفيذ جميع القواعد التي تُطبق بعدها.

اعتمدت القيمتان 100 و 200 كقيم تصميمية صريحة تضمن وجود فرق واضح في الأولوية بين نوعي القواعد، ولا تمثلان مطلب ثابت في ميفاق OpenFlow وإنما اختيرت لتوفير مجال لإضافة مستويات أولوية أخرى مستقبلاً.

3.2.6- نسيقة مراقبة التدفق (Flow Monitoring Module)

تُعد هذه النسيقة إحدى المكونات الأساسية في النظام، إذ تتولى مراقبة حركة السير التي تمر عبر مبدلات OVS وجمع الإحصائيات المتعلقة بقواعد التدفق المطبقة. تتميز بقدرتها على رصد الرزم التي تم حظرها بواسطة سياسات جدار النار، على الرغم من أن هذه الرزم لا تصل إلى متحكم Ryu بعد تثبيت قواعد الحظر على المبدلات.

– آلية العمل:

- إنشاء نيسب مستقل عند بدء تشغيل متحكم Ryu.
- إرسال طلب OFFFLOWSTATS REQUEST إلى جميع مبدلات OVS المتصلة بالمتحكم كل خمس ثوانٍ.
- يستجيب كل مبدلة بإرسال إحصائيات جميع قواعد التدفق المثبتة لديه، متضمنةً عدد الرزم (Packet Count)، وعدد البايتات (Byte Count) التي طابقت كل قاعدة.
- يقوم المتحكم بتحليل الإحصائيات المستلمة، ومقارنتها بالقيم السابقة لاكتشاف أي زيادة في عدد الرزم المطابقة لقواعد الحظر.
- عند اكتشاف تطابقات جديدة، يُسجل النظام الحدث في قاعدة المعطيات مع معلومات القاعدة والزمن وعدد الرزم المحظورة.
- تُستخدم هذه البيانات لاحقاً في إنشاء التقارير الأمنية وعرضها ضمن صفحة Monitoring في واجهة النظام.

يُمثل الشكل التالي آلية سير عملية المراقبة.



الشكل 9- سير عملية المراقبة (Monitoring).

يُمثِّل الشكل 9 آلية مراقبة الأحداث الأمنية الحاصلة ضمن الشبكة، بدءاً من إنشاء نيسب لتتبع الأحداث وصولاً إلى تسجيلها ضمن قاعدة المعطيات وعرضها في صفحة Monitoring ضمن الواجهة.

4.2.6- نَسِيقَةُ الواجهة (Interface Module)

تُعَدُّ نقطة التفاعل الرئيسية بين مدير النظام وجدار النار، حيث توفر واجهة رسومية سهلة الاستخدام تُمكن المسؤول من إدارة السياسات الأمنية ومراقبة حالة الشبكة دون الحاجة إلى التعامل المباشر مع أوامر نظام التشغيل أو متحكم SDN. طُورت هذه الخدمة باستخدام إطار العمل Flask، وتعمل كطبقة وسيطة بين المستخدم وبقية مكونات النظام، إذ تستقبل طلبات المستخدم، وتعالجها، ثم تتواصل مع قاعدة المعطيات ومتحكم Ryu لتنفيذ العمليات المطلوبة وعرض نتائجها بصورة آنية. تتكون الواجهة من مجموعة من الصفحات الرئيسية، يؤدي كل منها وظيفة محددة ضمن النظام.

1.4.2.6- لوحة التحكم (Dashboard)

تمثل لوحة التحكم الصفحة الرئيسية للنظام، حيث توفر نظرة شاملة على حالة جدار النار والشبكة. تعرض الصفحة إحصائيات آنية تساعد مدير النظام على تقييم أداء النظام ومتابعة نشاطه دون الحاجة إلى التنقل بين الصفحات المختلفة. تتضمن الصفحة ما يلي:

- عرض عدد القواعد الأمنية النشطة.
- عرض إجمالي الزم التي تم حظرها.
- عرض حالة متحكم Ryu.
- عرض رسوم بيانية خطية (Scalable Vector Graphics (SVG) توضح التغير التراكمي للأحداث الأمنية مع مرور الزمن.
- تحديث البيانات تلقائياً كل عشر ثوانٍ لضمان عرض أحدث المعلومات.

2.4.2.6- صفحة إدارة السياسات (Policies)

تُستخدم هذه الصفحة لإدارة سياسات جدار النار، حيث تتيح للمسؤول إنشاء السياسات الأمنية وتعديلها وحذفها من خلال واجهة رسومية بسيطة. توفر الصفحة الوظائف التالية:

- إضافة سياسة جديدة عبر نموذج إدخال تفاعلي.
- تحديد عنوان IP للمصدر والوجهة.
- اختيار الميفاق (TCP/UDP/ICMP).

- تحديد أرقام المنافذ عند الحاجة.
 - اختيار الإجراء المطلوب (Block/Allow).
 - تحديد أولوية القاعدة.
 - تعديل السياسات الحالية باستخدام نافذة حوار (Modal) لتقليل أخطاء الإدخال.
 - حذف السياسات مع طلب تأكيد قبل تنفيذ عملية الحذف.
- تُرسل جميع العمليات إلى متحكم Ryu بعد التحقق من صحة البيانات، ليتم تطبيقها مباشرةً على مبدلات OVS.

3.4.2.6- صفحة المراقبة (Monitoring)

تختص هذه الصفحة بعرض التقارير الأمنية وإحصائيات حركة السير الناتجة عن تطبيق سياسات جدار النار، مما يساعد مدير النظام على تقييم فعالية السياسات واكتشاف الأنماط غير الطبيعية في الشبكة. تتضمن الصفحة:

- رسومًا بيانية خطية (SVG) توضح تطور الأحداث الأمنية مع مرور الزمن.
- مخططات توضح توزيع الأحداث حسب نوع الحدث والميفاق المستخدم.
- جدولًا تفصيليًا للزعم المحجوبة يتضمن:
- الطابع الزمني.
- عنوان IP للمصدر.
- عنوان IP للوجهة.
- الميفاق.
- أرقام المنافذ.
- تحديث البيانات بصورة دورية لضمان عرض أحدث نتائج المراقبة.

4.4.2.6- صفحة السجلات (Event Logs)

توفر هذه الصفحة سجلًا زمنيًا شاملًا لجميع الأحداث التي يسجلها النظام أثناء تشغيله، مما يتيح تتبع العمليات التي تمت على السياسات ومراجعة الأحداث الأمنية السابقة. تشمل الصفحة:

- عرض جميع السجلات مرتبة زمنيًا.
- تصفية السجلات حسب نوع الحدث.

- عرض ملخصات إحصائية للأحداث.
- توفير سجل تدقيق (Audit Log) يساعد مدير النظام على مراجعة جميع العمليات المنفذة داخل النظام.

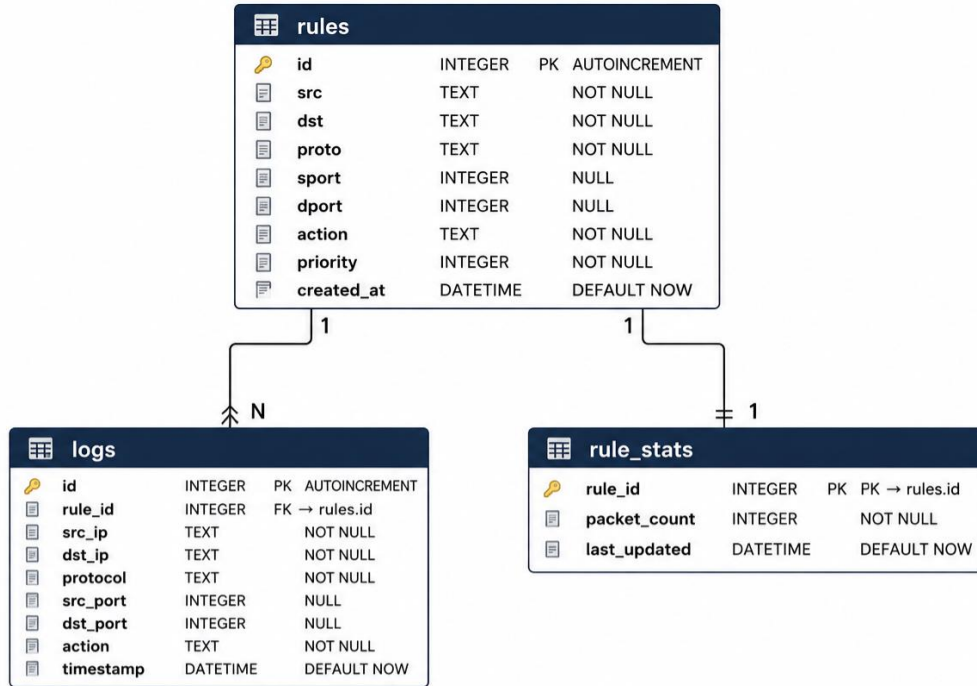
3.6- تصميم قاعدة المعطيات

يُعد تصميم قاعدة المعطيات أحد المكونات الأساسية للنظام، إذ يتولى حفظ جميع البيانات اللازمة لتشغيل جدار النار وإدارة سياساته وتتبع الأحداث الأمنية الناتجة عن تطبيقها. وقد اعتمدت قاعدة معطيات SQLite لما تتميز به من خفة الحجم وسهولة التكامل مع تطبيقات Python وعدم حاجتها إلى خادم مستقل لإدارة قواعد المعطيات، مما يجعلها مناسبة لتطبيقات الإدارة المحلية والأنظمة متوسطة الحجم.

اعتمد التصميم على مبدأ الفصل بين البيانات التشغيلية والبيانات الإحصائية وسجلات الأحداث، حيث تم تقسيم قاعدة المعطيات إلى ثلاثة جداول، يختص كل منها بوظيفة محددة، مع استخدام المبدلات الأساسية والمبدلات الأجنبية لضمان سلامة البيانات والمحافظة على التكامل المرجعي بينها.

1.3.6- مخطط الكيانات والعلاقات (Entity Relationship Diagram)

تتكون قاعدة المعطيات من ثلاثة جداول رئيسية هي rules و logs و rule_stats، كما يوضح الشكل 10. يمثل جدول rules المصدر الرئيس لجميع سياسات جدار النار، بينما يحتفظ جدول logs بسجل الأحداث الأمنية الناتجة عن تطبيق تلك السياسات، ويختص جدول rule_stats بحفظ الإحصائيات الخاصة بكل قاعدة، مثل عدد الرزم المطابقة وآخر وقت تم فيه تحديث الإحصائيات.



الشكل 10- مخطط الكيانات والعلاقات.

يُمثل الشكل 10 قاعدة المعطيات المستخدمة في النظام، والتي تتألف من ثلاث جداول مع بيان العلاقات بينها.

1.1.3.6- تصميم جدول القواعد (rules Table)

جدول rules هو الجدول الرئيس في قاعدة المعطيات، إذ يحتوي على جميع سياسات جدار النار المعرفة داخل النظام، ويُستخدم مرجعاً لبقية الجداول. يُمثل كل سجل داخل هذا الجدول قاعدة أمنية مستقلة يمكن تطبيقها على مبدلات OVS بواسطة متحكم Ryu.

يتضمن الجدول مجموعة من الحقول التي تصف خصائص القاعدة الأمنية، وتشمل عنوان IP للمصدر، وعنوان IP للوجهة، ونوع الميفاق، وأرقام المنافذ المصدر والوجهة عند الحاجة، بالإضافة إلى الإجراء المطلوب تنفيذه (السماح أو الحظر)، ومستوى أولوية القاعدة، وتاريخ إنشائها.

يُستخدم الحقل id كمفتاح أساسي (Primary Key) لتمييز كل قاعدة بصورة فريدة، كما تعتمد الجداول الأخرى عليه لإقامة العلاقات المرجعية، مما يضمن إمكانية تتبع جميع العمليات والإحصائيات المرتبطة بكل قاعدة.

2.1.3.6- تصميم جدول السجلات (logs Table)

يختص جدول logs بحفظ جميع الأحداث الأمنية الناتجة عن تطبيق سياسات جدار النار أثناء تشغيل النظام، ويشكل سجلاً زمنياً يمكن الرجوع إليه لمراجعة العمليات وتحليل نشاط الشبكة.

يحتوي الجدول على معلومات تفصيلية عن كل حدث، مثل عنواني IP للمصدر والوجهة، والميفاق المستخدم، وأرقام المنافذ، والإجراء الذي اتخذه جدار النار، بالإضافة إلى الطابع الزمني الذي يحدد وقت وقوع الحدث.

يرتبط هذا الجدول بجدول rules من خلال الحقل rule_id الذي يمثل مفتاحاً أجنبياً (Foreign Key)، حيث يمكن أن ترتبط القاعدة الأمنية الواحدة بعدد كبير من السجلات الناتجة عن تطبيقها، وهو ما يمثل علاقة واحد إلى متعدد (One-to-Many) بين الجدولين.

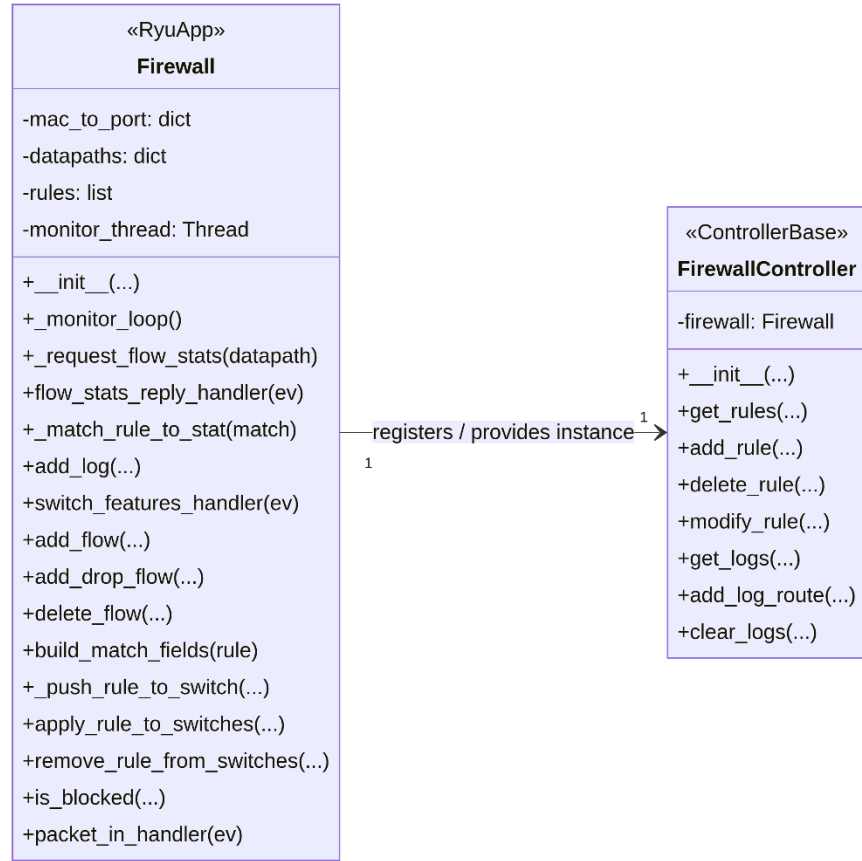
3.1.3.6- تصميم جدول إحصائيات التدفق (rule_stats Table)

يُستخدم جدول rule_stats لتخزين الإحصائيات الخاصة بكل قاعدة من قواعد جدار النار، مما يسمح بمتابعة مدى استخدام القواعد وقياس فعاليتها أثناء تشغيل النظام.

يحتوي الجدول على عدد الرزم التي طابقت كل قاعدة (packet_count) بالإضافة إلى وقت آخر تحديث للإحصائيات (last_updated). تعتمد هذه البيانات على إحصائيات التدفق المسترجعة دورياً من مبدلات OVS بواسطة متحكم Ryu باستخدام رسائل OFPFlowStatsRequest. يرتبط هذا الجدول بجدول rules بعلاقة واحد إلى واحد (One-to-One)، إذ يمتلك كل سجل في جدول rules سجلاً واحداً فقط في جدول rule_stats، ويُستخدم الحقل rule_id كمفتاح أساسي ومفتاح أجنبي في الوقت نفسه لضمان وجود إحصائيات مرتبطة بكل قاعدة وعدم تكرارها.

4.6- مخطط الصفوف (Class Diagram)

يوضح مخطط الصفوف البنية الكائنية للتطبيق من خلال بيان الصفوف البرمجية الرئيسة والعلاقات التي تربط بينها، مما يساهم في فهم توزيع المسؤوليات وآلية التعاون بين مكونات النظام أثناء التنفيذ.



الشكل 11- مخطط الصفوف للنظام.

يوضح الشكل 11 علاقة التسجيل بين صف Firewall الذي يحتوي على منطق التحكم بالشبكة ومعالجة الرزم، وصف FirewallController المسؤول عن واجهة REST API؛ حيث يقوم Firewall بتسجيل FirewallController عند بدء التشغيل وتزويده بمراجع لنسخته الخاصة (بعلاقة One-to-one) ليتمكن الأخير من الوصول إلى القواعد وتطبيقها على المبدلات.

استعرض هذا الفصل التصميم التفصيلي للنظام، موضحاً بنية مكّوناته وآلية تكاملها، بالإضافة إلى تصميم قاعدة المعطيات ووحدات النظام ومخطط الصفوف. يشكّل هذا التصميم الأساس الذي استندت إليه عملية تنجيز النظام، والتي سيتم تناولها بالتفصيل بعد التكلم عن الأدوات المستخدمة في التنجيز.

الفصل السابع

الأدوات المستخدمة

يعرض هذا الفصل الأدوات المستخدمة لتنفيذ المشروع.

1.7- متحكم Ryu

يمثل Ryu إطار عمل مفتوح المصدر مكتوب بلغة Python لتطوير تطبيقات SDN، ويدعم ميفاقات OpenFlow المختلفة، وخاصة الإصدار 1.3. يعتمد على نموذج مُقَوَّد بالأحداث (Event-Driven)، حيث يتفاعل مع الرسائل الواردة من المبدلات عبر متوَلِّيات أحداث (Event Handlers). كما يوفر واجهة REST API تسمح للتطبيقات الخارجية بإضافة أو حذف قواعد OpenFlow باستخدام طلبات HTTP، وهو ما استُخدم في هذا المشروع لربط واجهة Flask بالمتحكم.

2.7- Open vSwitch (OVS)

مبدلة افتراضية مفتوحة المصدر تدعم ميفاق OpenFlow، وتُستخدم بصورة واسعة في بيئات SDN والمحاكاة الافتراضية.

3.7- لغة البرمجة Python

تُعدّ إحدى أكثر لغات البرمجة استخداماً في تطبيقات الشبكات والأمن السيبراني، لما تتميز به من بساطة ووفرة المكتبات البرمجية. استُخدمت في تطوير جميع المكونات البرمجية للنظام، بما في ذلك متحكم Ryu، وتطبيق Flask، وتكنولوجيا Containernet، إضافةً إلى التعامل مع قاعدة المعطيات.

4.7- إطار العمل Flask

إطار عمل وب خفيف الوزن يعتمد على لغة Python، ويوفر الأدوات الأساسية اللازمة لبناء تطبيقات الويب وواجهات REST دون فرض بنية معقدة على المشروع.

5.7 -HTML5, CSS3, JavaScript

استُخدمت هذه التقنيات لتطوير الواجهة الأمامية للنظام، حيث تولت HTML بناء هيكل الصفحات، بينما استُعملت CSS لتنسيق الواجهة وتحسين تجربة المستخدم، في حين كان الهدف من JavaScript هو تنفيذ العمليات التفاعلية وتحديث البيانات دون الحاجة إلى إعادة تحميل الصفحات. وقد اعتمد المشروع على JavaScript الأصلي (Vanilla JavaScript) دون استخدام أي إطار عمل خارجي، وذلك لتقليل التبعيات والمحافظة على بساطة التطبيق.

6.7 -Scalable Vector Graphics (SVG)

تقدم الإمكانية لإنشاء الرسوم البيانية داخل صفحات النظام، لما توفره من إمكانية رسم أشكال عالية الجودة يمكن تكبيرها أو تصغيرها دون فقدان الدقة، إضافة إلى سهولة تحديثها ديناميكياً باستخدام JavaScript.

7.7 -Containernet

يُعدّ امتداداً لمحاكي Mininet، ويتيح إنشاء بيئات SDN تستخدم حاويات Docker بوصفها مضيفون (Hosts) حقيقية، الأمر الذي يوفر بيئة اختبار أكثر واقعية لإجراء سيناريوهات مختلفة، بما في ذلك محاكاة الهجمات الأمنية.

8.7 -Docker

هي منصة مفتوحة المصدر مصممة لأتمتة عملية نشر التطبيقات وتوسيع نطاقها (Scaling) وإدارتها في حاويات خفيفة الوزن (Containers). الحاوية هي بيئة معزولة تجمع بين التطبيق وجميع تبعياته، مما يسمح بتشغيله بشكل متسق عبر بيئات حوسبة مختلفة، مثل أنظمة تشغيل مختلفة أو مزودي الخدمات السحابية. وفّر Docker بيئة تشغيل موحدة وقابلة لإعادة الاستخدام، وساعد في تنفيذ خدمات الشبكة وسيناريوهات الاختبار بصورة مستقرة.

9.7 -SQLite

اعتمد المشروع على SQLite بوصفها قاعدة معطيات علائقية خفيفة الوزن لا تتطلب خادم قواعد بيانات مستقل، مما يجعلها مناسبة للتطبيقات المحلية ومتوسطة الحجم.

10.7- بيئة التطوير

اعتمد تطوير المشروع على نظام التشغيل Ubuntu 20.4.06 لما يوفره من توافق كامل مع أدوات SDN المستخدمة، كما استخدم Oracle VirtualBox لتشغيل بيئة التطوير داخل آلة افتراضية، في حين استخدمت بيئات Python الافتراضية (Virtual Environments) لعزل التبعيات الخاصة بمكونات النظام المختلفة ومنع تعارضها.

استعرض هذا الفصل الأدوات والتقنيات البرمجية التي اعتمدنا عليها أثناء تنجيز النظام، وهو ما سنتكلم عنه في الفصل القادم.

الفصل الثامن

تنجيز النظام

يُعرض هذا الفصل كيفية تنجيز النظام مع تفصيل كل جزء من أجزائه.

1.8- مقدمة

يتناول هذا الفصل آلية تنجيز نظام جدار النار المبني على SDN، موضحاً كيفية تنفيذ مكونات النظام المختلفة وتحويل التصميم المقترح إلى نظام عملي متكامل. اعتمد تنفيذ النظام على متحكم Ryu بوصفه العنصر المركزي لإدارة الشبكة، وتطبيق Flask لتطوير واجهة المستخدم، وقاعدة معطيات SQLite لتخزين السياسات والسجلات الأمنية. يستعرض هذا الفصل آلية عمل كل نسيقة، ومسار تنفيذ العمليات الأساسية، والتفاعل فيما بينها لتحقيق إدارة مركزية وفعالة لجدار النار.

2.8- تنجيز نسيقة المتحكم

1.2.8- مقدمة

تُنفَّذ هذه النسيقة باستخدام إطار العمل Ryu الذي يعتمد نموذج البرمجة المقوّد بالأحداث، مما يسمح بالاستجابة الفورية لأحداث الشبكة وإدارة المبدّلات بصورة ديناميكية.

2.2.8- تفاصيل التنجيز

1.2.2.8- إعادة تحميل السياسات

يعتمد النظام على إعادة قراءة جميع السياسات المخزنة في قاعدة المعطيات فور تشغيل المتحكم، ثم إعادة نشرها تلقائياً على كل مبدّلة يتصل بالشبكة، مما يضمن استمرارية عمل جدار النار دون تدخل مدير النظام.

2.2.2.8- معالجة اتصال المبدلات

عند اتصال مبدلة جديد بالمتحكم تُثبت القاعدة الافتراضية أولاً، ثم تُرسل جميع سياسات جدار النار المخزنة إلى المبدلة وفق ترتيب الأولويات المحدد. يضمن ذلك انضمام أي مبدلة جديد إلى الشبكة مع تطبيق جميع السياسات الأمنية مباشرة.

3.8- تنجيز نسيقة إدارة القواعد

1.3.8- مقدمة

تم بناء هذه النسيقة باستخدام لغة Python ضمن إطار العمل Ryu؛ لأنّ Ryu يعتمد هذه اللغة بشكل أساسي لتطوير تطبيقات التحكم بـ SDN، ويوفر واجهات برمجية جاهزة للتعامل مع ميفاق OpenFlow 1.3، مما سهّل تنفيذ عمليات إنشاء السياسات الأمنية وتحويلها إلى قواعد تدفق وإدارتها بصورة مباشرة.

2.3.8- تفاصيل التنجيز

1.2.3.8- خوارزمية إضافة سياسة جديدة

تبدأ العملية بالتحقق من صحة المدخلات، ثم تُحفظ السياسة داخل قاعدة المعطيات، وبعد ذلك يُنشأ سجل خاص بإحصائيات هذه القاعدة، ثم تُحوّل السياسة إلى قاعدة تدفق (Flow Rule) متوافقة مع OpenFlow قبل نشرها على جميع المبدلات، وأخيراً يُسجل الحدث ضمن السجل الأمني.

2.2.3.8- خوارزمية تعديل سياسة

يقوم النظام بإزالة قاعدة التدفق القديمة أولاً، ثم تحديث بيانات السياسة داخل قاعدة المعطيات، وبعد ذلك تُبنى قاعدة تدفق جديدة وفق القيم المعدلة وتُنشر على جميع المبدلات، ثم يُسجل الحدث في السجل الأمني.

3.2.3.8- خوارزمية حذف سياسة

عند حذف سياسة أمنية، لا يقتصر الأمر على إزالة بياناتها من قاعدة المعطيات، وإنما تُزال قاعدة التدفق المناظرة لها أولاً من جميع المبدلات المتصلة، ثم تُحذف بياناتها وإحصائياتها، وأخيراً يُسجل حدث الحذف في سجل الأحداث الأمنية.

4.2.3.8- بناء حقول المطابقة (Flow Match Construction)

تُبنى حقول المطابقة بصورة ديناميكية اعتماداً على المعلومات التي يحددها مدير النظام، بحيث تُضاف فقط الحقول اللازمة لنوع الميفاق المستخدم، فعند اختيار ميفاق TCP أو UDP تُضاف حقول المنافذ بالإضافة إلى عناوين IP، أما عند استخدام ميفاق ICMP فلا تُستخدم حقول المنافذ لعدم وجودها في هذا الميفاق. يسمح هذا الأسلوب ببناء قواعد تدفق دقيقة دون إضافة حقول غير ضرورية، الأمر الذي يقلل حجم قواعد OpenFlow ويرفع كفاءة المطابقة داخل المبدلات.

5.2.3.8- آلية تحديد الأولويات

عند استلام سياسة جديدة، يتحقق النظام من نوع الإجراء المحدد، ثم يُسند قيمة الأولوية المناسبة تلقائياً. بعد تحديد قيمة الأولوية، تُضمّن هذه القيمة ضمن رسالة OFPFlowMod المرسلة إلى مبدلات OVS، لتُخزّن داخل جدول التدفق (Flow Table). عند وصول رزمة تنطبق على أكثر من قاعدة، تتولى المبدلة اختيار القاعدة ذات الأولوية الأعلى وتنفيذها مباشرةً وفق آلية عمل ميفاق OpenFlow، دون الحاجة إلى أي معالجة إضافية من المتحكم.

4.8- تنجيز نسيقة مراقبة التدفق

1.4.8- مقدمة

تم تطوير نسيقة مراقبة التدفقات باستخدام Python بالاعتماد على إمكانيات Ryu الخاصة بجمع إحصائيات التدفق، حيث تعمل بصورة دورية ضمن نيسب تنفيذ مستقل (Background Thread) لاستطلاع إحصائيات قواعد التدفق وتحليلها بهدف رصد الرزم المحجوبة وتحديث السجلات الأمنية.

2.4.8- تفاصيل التنجيز

1.2.4.8- مشكلة مراقبة قواعد الحجب

عند تثبيت قاعدة Drop داخل مبدلة OVS، تُحجّب الرزم مباشرةً داخل المبدلة. بالتالي، لا تصل إلى المتحكم، الأمر الذي يجعل الاعتماد على رسائل Packet-In غير ممكن لرصد عمليات الحجب. لذلك اعتمدنا في تنجيز النظام على إحصائيات التدفق التي يوفرها ميفاق OpenFlow.

2.2.4.8- خوارزمية مراقبة الرزم المحجوبة

يتم العمل بشكل دوري (كل 5 ثوانٍ)، حيث ترسل هذه النسيقة طلباً للحصول على إحصائيات التدفق من جميع المبدلات. بعد استلام الردود، تُقارن قيمة عداد الرزم الحالي بالقيمة المخزنة سابقاً لكل قاعدة حجب. عند اكتشاف زيادة في العداد، يستنتج النظام حدوث عملية حجب جديدة، فيسجل الحدث داخل قاعدة المعطيات ثم يُحدّث قيمة العداد استعداداً لدورة المراقبة التالية.

5.8- تنجيز نسيقة الواجهة

1.5.8- مقدمة

تم بناء واجهة المستخدم باستخدام إطار العمل Flask للواجهة الخلفية، وتقنيات HTML5 و CSS3 و JavaScript للواجهة الأمامية.

2.5.8- تفاصيل التنجيز

1.2.5.8- آلية الاتصال بين Flask و Ryu

اعتمد النظام فصل واجهة المستخدم عن متحكم الشبكة، حيث يعمل تطبيق Flask وسيطاً بين المتصفح ومتحكم Ryu، فعند تنفيذ أي عملية من واجهة المستخدم، يستقبل Flask الطلب ويحوّله إلى طلب REST موجه إلى المتحكم، وبعد تنفيذ العملية تُعاد النتيجة إلى Flask ليتم عرضها للمستخدم.

2.2.5.8- تحديث البيانات

اعتمدت الواجهة أسلوب التحديث الدوري (Polling)، حيث تُرسل طلبات دورية إلى المتحكم لتحديث الإحصائيات والرسوم البيانية والأحداث الأمنية المعروضة للمستخدم.

6.8- تنجيز قاعدة المعطيات

1.6.8- مقدمة

تُجَزّت قاعدة المعطيات باستخدام نظام إدارة قواعد المعطيات SQLite المدمج مع لغة Python عبر مكتبة sqlite3.

2.6.8- تفاصيل التنفيذ

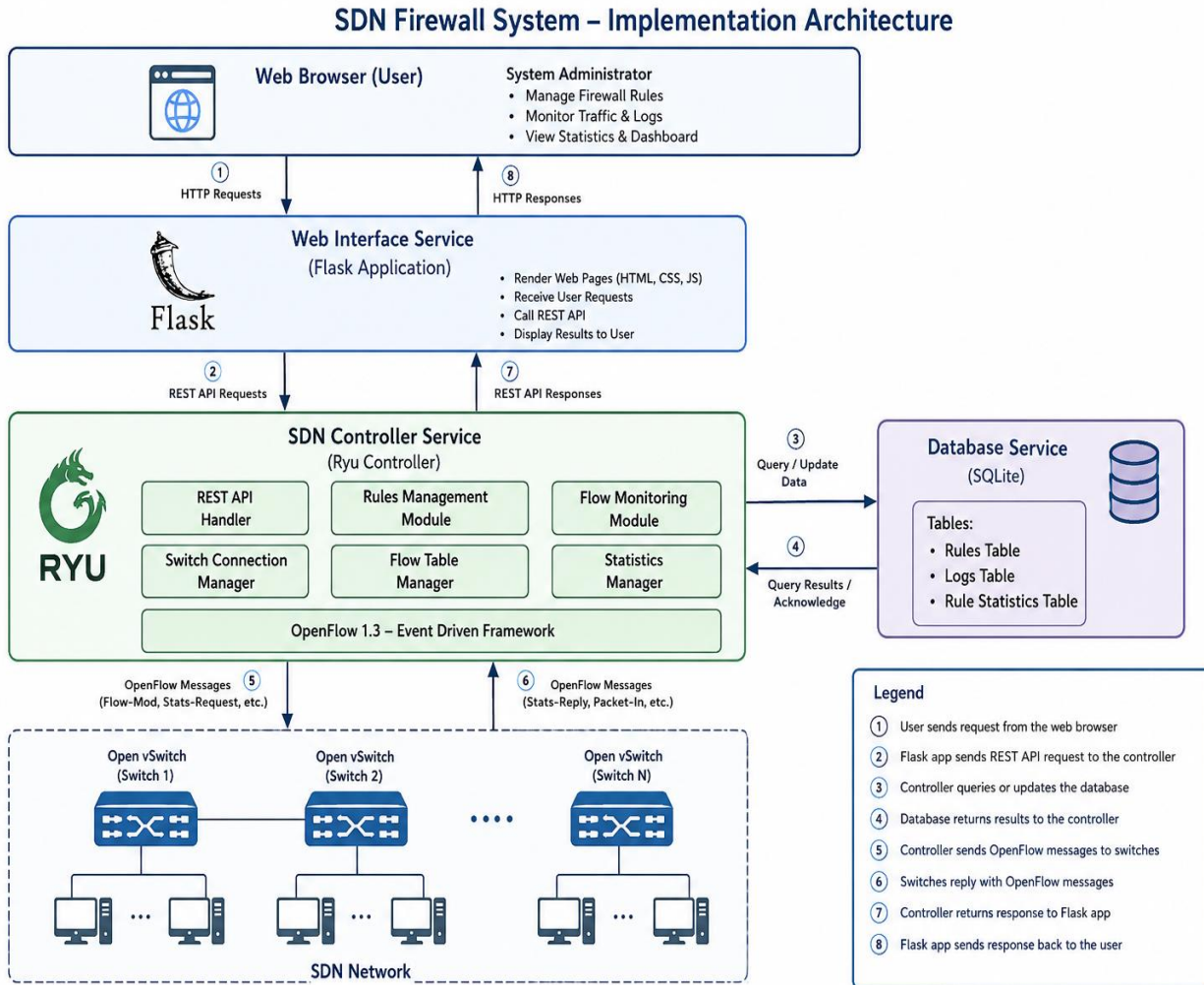
1.2.6.8- إنشاء قاعدة المعطيات

عند تشغيل النظام لأول مرة، تُنشأ الجداول المطلوبة تلقائياً إذا لم تكن موجودة مسبقاً، مما يسمح بتشغيل النظام مباشرة دون الحاجة إلى تهيئة يدوية. يتضمن هذا الأسلوب إمكانية إعادة تشغيل المتحكم في أي وقت مع المحافظة على جميع البيانات المخزنة سابقاً.

2.2.6.8- آلية التنظيف التلقائي للسجلات

مع استمرار تشغيل النظام يزداد عدد الأحداث الأمنية بصورة مستمرة، الأمر الذي يؤدي إلى تضخم قاعدة البيانات وانخفاض كفاءة عمليات الاستعلام. لهذا اعتمدت آلية تنظيف تلقائي تعتمد على حد أعلى لعدد السجلات المخزنة (10,000 سجل).

7.8- مخطط النظام التنفيذي



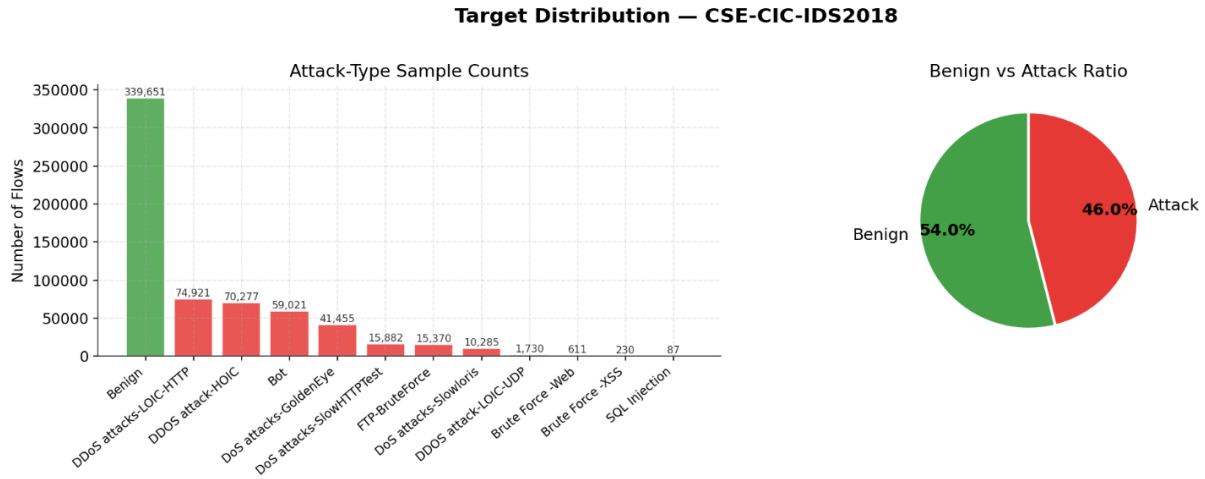
الشكل 12- التكامل بين مكونات النظام.

يوضح الشكل 12 آلية تكامل النسيقات البرمجية للنظام، بدءاً من تفاعل مدير النظام مع واجهة المستخدم، مروراً بنسيقة المتحكم وإدارة القواعد وقاعدة المعطيات، وانتهاءً بتطبيق سياسات جدار النار على مبدلات الشبكة ومراقبة الأحداث الأمنية.

8.8- تقييم خوارزميات تعلم الآلة لتصنيف الهجمات

1.8.8- مجموعة البيانات (Dataset)

اعتمد المشروع على مجموعة بيانات CSE-CIC-IDS2018، وهي مجموعة بيانات مرجعية في مجال أنظمة كشف التسلل [11]. تحتوي المجموعة على تسجيلات تدفقات شبكية ثنائية الاتجاه (Bidirectional Network Flows) جُمعت على مدار عشرة أيام، وتغطي حركة مرور سليمة (Benign) إضافة إلى 13 نوعاً مختلفاً من الهجمات مثل هجمات DoS، FTP-BruteForce، DDoS، وغيرها. يُوصف كل تدفق شبكي (Flow) بأكثر من 80 سمة إحصائية (طول الرزم، فترات الوصول بين الرزم، عدد البايتات، رايات TCP، وغيرها). تم تحميل عشرة ملفات CSV (يوم واحد لكل ملف)، مع أخذ عينة قدرها 75,000 صف من كل ملف لأغراض التطوير، ليصل الحجم الإجمالي بعد إزالة التكرارات إلى نحو 629,521 تدفقاً، موزعة بنسبة تقارب 54% سليمة مقابل 46% هجمات كما هو موضح في الشكل 13، وهو توازن جيد نسبياً بين الفئتين.



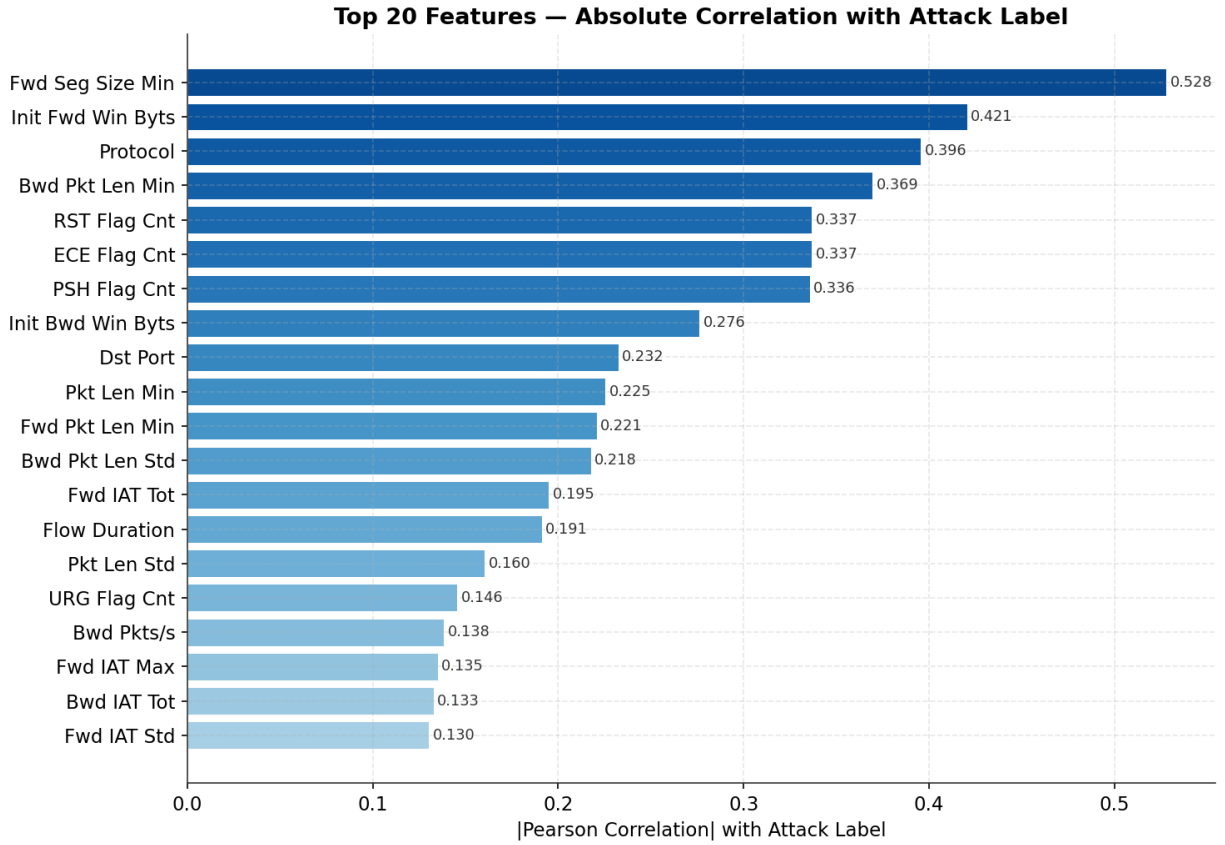
الشكل 13- توزيع أنواع الهجمات في مجموعة البيانات.

يوضح الشكل 13 نسبة عينات كل هجوم على حدة، إضافةً إلى النسبة الإجمالية بين حركة السير السليمة والهجمات.

2.8.8- المعالجة المسبقة للبيانات (Preprocessing) وهندسة السمات (Feature Engineering)

قبل البدء بعملية تدريب نماذج تعلم الآلة، خضعت مجموعة البيانات إلى سلسلة من خطوات المعالجة المسبقة وهندسة السمات بهدف تحسين جودة البيانات، وتقليل الضجيج، ومنع حدوث تسرب البيانات (Data Leakage) بين مجموعات التدريب والتحقق والاختبار. وقد نُفذت هذه الخطوات بالترتيب الآتي:

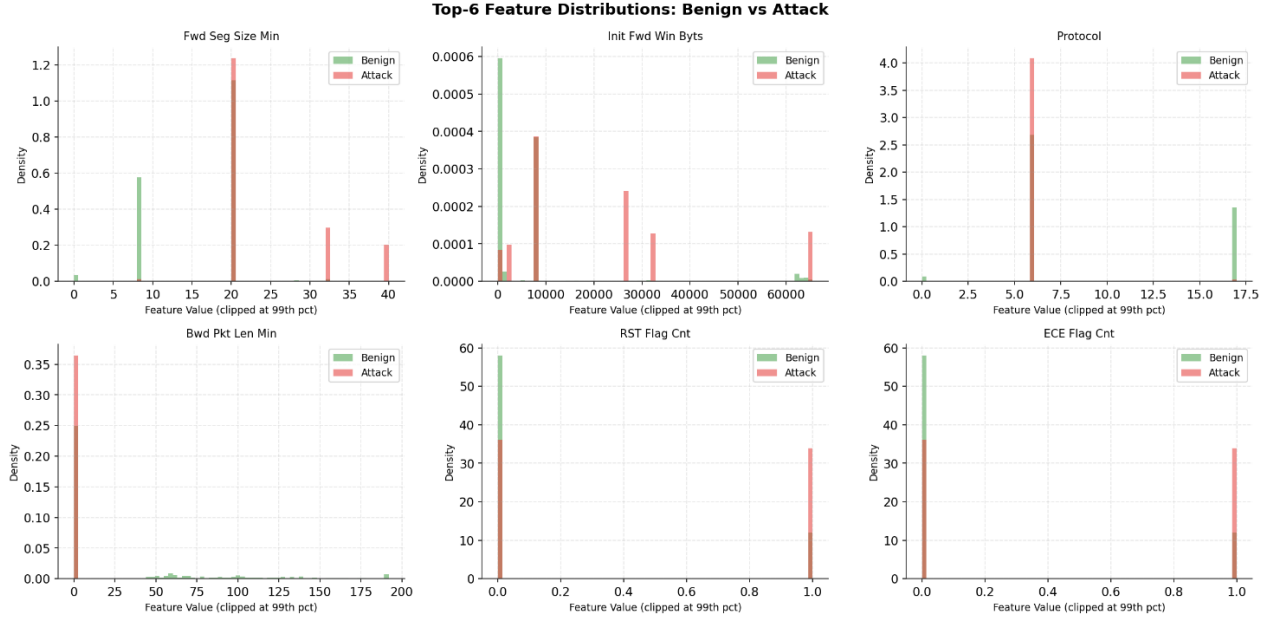
1. تنظيف البيانات (Data Cleaning): أزيلت جميع التسجيلات المكررة تماماً، كما حُذفت الصفوف الناتجة عن تكرار عناوين الأعمدة داخل بعض ملفات البيانات، وذلك لضمان اتساق البيانات المستخدمة في التدريب.
 2. المعالجة الزمنية: حُوِّل حقل الختم الزمني (Timestamp) إلى صيغة مناسبة، ثم رُتبت التسجيلات ترتيباً زمنياً تصاعدياً للحفاظ على التسلسل الحقيقي للبيانات أثناء عملية التقسيم.
 3. إنشاء التصنيف الثنائي (Binary Classification): تم تحويل فئات البيانات إلى تصنيف ثنائي، بحيث مُثِّلت الحركة الطبيعية بالقيمة 0، بينما مُثِّلت جميع أنواع الهجمات بالقيمة 1، بهدف تحويل المشكلة إلى مسألة تصنيف ثنائي.
 4. معالجة القيم المفقودة وغير الصالحة: حُوِّلَت جميع السمات إلى قيم رقمية، واستُبدلت القيم اللاهائية والقيم المفقودة بقيم مناسبة، ثم تحويلها باستخدام الوسيط (Median) المحسوب من مجموعة التدريب فقط؛ وذلك لتجنب انتقال أي معلومات من مجموعتي التحقق والاختبار إلى مرحلة التدريب.
 5. اختيار السمات ذات التباين المناسب: استُخدمت تقنية عتبة التباين (Variance Threshold) لاستبعاد السمات ذات التباين المنخفض أو شبه الثابت؛ لأنها لا تضيف معلومات مؤثرة إلى عملية التصنيف، مما أدى إلى تقليص عدد السمات والإبقاء على 68 سمة.
 6. إزالة السمات عالية الارتباط: طُبِّقت تقنية Correlation Pruning لحذف إحدى السمات من كل زوج تتجاوز قيمة معامل الارتباط المطلق بينهما 0.95، وذلك للحد من مشكلة التعدد الخطي (Multicollinearity) وتقليل التكرار في البيانات، لينخفض عدد السمات إلى 46 سمة.
- فيما يلي الشكل 14 الذي يمثل أعلى 20 سمة ارتباطاً بالتصنيف هجمة (Attack).



الشكل 14- أعلى 20 سمة ارتباطاً بالتصنيف هجمة.

يعرض الشكل 14 السمات العشرين الأكثر ارتباطاً بتصنيف الهجوم، مرتبة تنازلياً حسب القيمة المطلقة لمعامل ارتباط Pearson.

كما يعرض الشكل 15 توزيع أفضل السمات للفصل بين الفئتين سليم (Benign) وهجوم (Attack).



الشكل 15- توزيع أفضل 6 سمات (سليم مقابل هجوم).

يقارن الشكل 15 توزيع قيم أهم ست سمات إحصائية بين الحركة الطبيعية والحركة الهجومية، بهدف إبراز مدى قدرة كل سمة على التمييز بين الفئتين.

7. ضبط أبعاد البيانات (Scaling): أخيراً، استُخدم StandardScaler لتوحيد مقياس جميع السمات بحيث يصبح متوسطها صفراً وتباينها يساوي واحداً. تم حساب معاملات ضبط الأبعاد اعتماداً على مجموعة التدريب فقط، ثم استُخدمت لتحويل مجموعتي التحقق والاختبار، حفاظاً على سلامة المنهجية ومنع تسرب البيانات.

بهذه الخطوات أصبحت البيانات أكثر جودة واتساقاً، كما أصبحت السمات المستخدمة في التدريب أكثر قدرة على تمثيل الأنماط الحقيقية، مما يسهم في تحسين أداء نماذج تعلم الآلة ورفع موثوقية نتائج التقييم.

3.8.8- الخوارزميات المستخدمة

لتقييم أداء نماذج تعلم الآلة في تصنيف حركة السير الشبكية إلى حركة طبيعية أو هجومية، تم تدريب واختبار خمس خوارزميات تنتمي إلى فئتين رئيسيتين، هما: التعلم العميق (Deep Learning) وتعلم الآلة القائم على المجاميع (Ensemble Learning)، وذلك بهدف مقارنة أدائها واختيار النموذج الأكثر كفاءة.

1.3.8.8- الشبكات العصبونية التلافيفية أحادية البعد (1-Dimensional Convolutional Neural Networks)

اعتمد نموذج CNN-1D على معالجة متجه السمات باعتباره إشارة أحادية البعد، وتكوّن من كتلتين تلافيفيتين (Convolutional Blocks). احتوت الكتلة الأولى على 32 مرشحاً (Filters) بحجم نواة (Kernel Size) 3، بينما احتوت الكتلة الثانية على 64 مرشحاً بالحجم نفسه. أُتبعَت كل كتلة بطبقة Batch Normalization لتحسين استقرار التدريب، ثم بطبقة Dropout لتقليل ظاهرة فرط التخصيص (Overfitting).

بعد ذلك، استُخدمت طبقة Flatten لتحويل السمات إلى متجه أحادي البعد مع الحفاظ على ترتيب السمات المستخرجة، ثم أُضيفت طبقة كثيفة (Dense Layer) مكوّنة من 64 عصبوناً باستخدام دالة التنشيط ReLU، تلتها طبقة إخراج تحتوي على عصبون واحد بدالة Sigmoid لإجراء التصنيف الثنائي.

تم تصميم هذه البنية لتمكين النموذج من اكتشاف الأنماط المحلية والعلاقات بين السمات المتجاورة التي قد تتغير بصورة متزامنة أثناء الهجمات الشبكية.

2.3.8.8- شبكة الذاكرة طويلة وقصير الأمد (Long Short-Term Memory)

اعتمد نموذج LSTM على طبقة واحدة من خلايا LSTM بحجم حالة مخفية (Hidden Size) مقداره 64 عصبوناً، والتي تستخدم بوابات النسيان (Forget Gate) والإدخال (Input Gate) والإخراج (Output Gate) لإدارة انتقال المعلومات داخل الذاكرة.

تم استخدام Layer Normalization على المخرجات لتحسين استقرار التدريب، ثم مرّت البيانات عبر رأس تصنيف (Classification Head) مكوّن من طبقتين كثيفتين؛ الأولى تضم 64 عصبوناً والثانية 32 عصبوناً، وكلاهما يستخدم دالة ReLU، قبل الوصول إلى طبقة الإخراج النهائية المكوّنة من عصبون واحد بدالة Sigmoid لتحديد ما إذا كانت حركة السير طبيعية أم هجومية.

اختيرت هذه البنية لقدرتها على تعلّم العلاقات المعقدة بين السمات والاحتفاظ بالمعلومات المهمة أثناء عملية التصنيف.

3.3.8.8- الغابة العشوائية (Random Forest)

يتكون نموذج Random Forest من 300 شجرة قرار (Decision Trees)، حيث تم تدريب كل شجرة على عينة Bootstrap مستقلة من بيانات التدريب، مع اختيار مجموعة عشوائية من السمات عند كل عملية انقسام داخل الشجرة.

يعتمد النموذج على دمج تنبؤات جميع الأشجار باستخدام التصويت بالأغلبية (Majority Voting) لإنتاج القرار النهائي، الأمر الذي يقلل من التباين ويرفع قدرة النموذج على التعميم ويحد من ظاهرة فرط التخصيص.

4.3.8.8- خوارزمية LightGBM

يعتمد نموذج LightGBM على Gradient Boosting لبناء سلسلة من أشجار القرار بطريقة Leaf-wise Growth، حيث يُختار في كل خطوة الورقة التي تحقق أكبر انخفاض في دالة الخطأ.

استُخدمت آلية Histogram-based Learning لتقليل استهلاك الذاكرة وتسريع التدريب، كما استُخدمت تقنية الإيقاف المبكر (Early Stopping) لإيقاف التدريب تلقائياً عند توقف تحسن أداء النموذج على مجموعة التحقق، مما يحد من فرط التخصيص ويحسن القدرة على التعميم (Generalization).

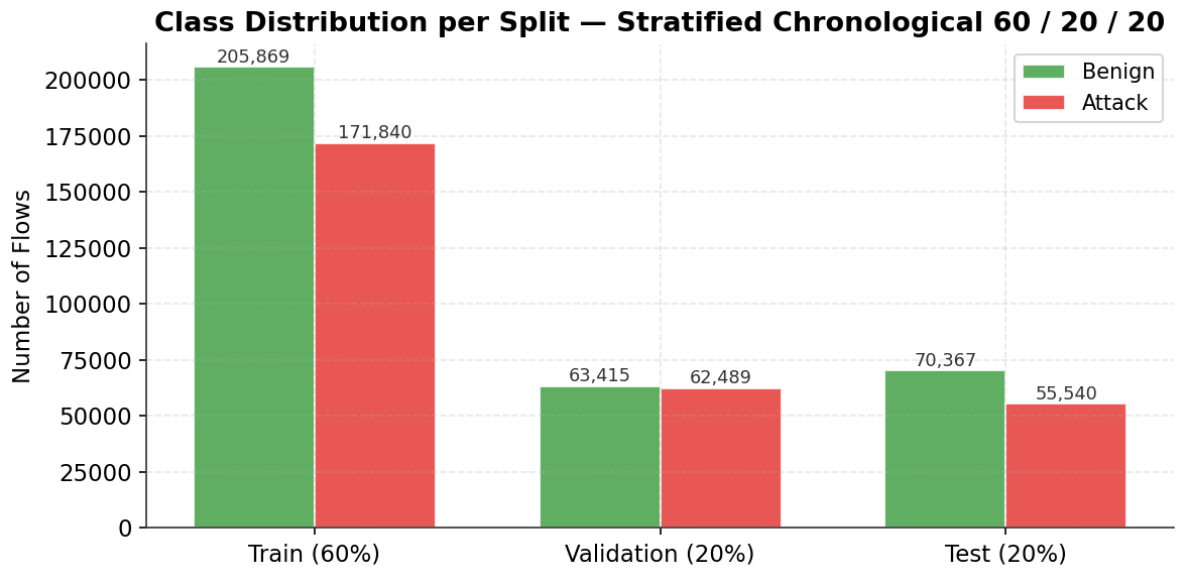
5.3.8.8- خوارزمية XGBoost

في نموذج XGBoost، يتم بناء أشجار القرار تدريجياً وفق أسلوب Gradient Boosting، مع استخدام طريقة Depth-wise Boosting لتوسيع الأشجار تدريجياً حسب العمق.

واستُخدمت الخوارزمية دالة هدف تتضمن معاملات تنظيم L1/L2 Regularization للحد من تعقيد النموذج، بالإضافة إلى استخدام معامل scale_pos_weight لمعالجة عدم توازن الفئات في مجموعة البيانات. كما استُخدمت تقنية Early Stopping.

4.8.8- منهجية التدريب والاختبار

اعتمد في هذه الدراسة أسلوب التقسيم الزمني للبيانات (Time-Based Splitting) لضمان محاكاة بيئة التشغيل الواقعية ومنع انتقال المعلومات المستقبلية إلى مرحلة التدريب. قُسمت البيانات الخاصة بكل يوم من أيام جمع البيانات بصورة مستقلة إلى 60% للتدريب (Training)، و20% للتحقق (Validation)، و20% للاختبار (Testing) كما هو موضح بالشكل 16، ثم جُمعت الأجزاء المناظرة لتكوين المجموعات النهائية. أسهم هذا الأسلوب في المحافظة على التسلسل الزمني للبيانات، مع الحفاظ في الوقت نفسه على التوزيع النسبي بين الحركة الطبيعية والهجمات في جميع المجموعات.



الشكل 16- توزيع الفئات على تقسيم 60/20/20.

يمثل الشكل 16 عدد العينات السليمة (باللون الأخضر) والهجومية (باللون الأحمر) في كل من مجموعات التدريب والتحقق والاختبار بعد التقسيم الزمني المقترح.

تم استخدام مجموعة التدريب في تدريب جميع نماذج تعلّم الآلة، بالإضافة إلى احتساب معاملات جميع عمليات المعالجة المسبقة، مثل تعويض القيم المفقودة وتقييس السمات، أما مجموعة التحقق فقد استخدمت لضبط معاملات النماذج واختيار أفضل إعداداتها، بما في ذلك تحديد عتبة القرار (Decision Threshold) وتطبيق آلية Early Stopping للحد من ظاهرة فرط التخصيص. في المقابل، احتُفظ بمجموعة الاختبار معزولةً طوال مراحل التدريب والضبط، ولم تُستخدم إلا في مرحلة التقييم النهائي للحصول على تقدير موضوعي لأداء النماذج.

بالنسبة إلى نماذج التعلم العميق، فقد دُرِّبَت لمدة لا تتجاوز 25 حقبة تدريبية (Epochs)، مع تطبيق آلية Early Stopping عند عدم تحسن أداء النموذج لمدة خمس حقوب متتالية، بالإضافة إلى استخدام تقليم التدرجات (Gradient Clipping) للحد من مشكلة انفجار التدرجات (Gradient Explosion)، وجدولة معدل التعلم (Learning Rate Scheduling) لتحسين عملية التقارب. أما نماذج تعلم الآلة القائمة على الأشجار، فقد اعتمدت فيها آلية موازنة أوزان الفئات (Class Weighting) لمعالجة عدم توازن البيانات، إلى جانب Early Stopping للحصول على أفضل أداء ممكن على مجموعة التحقق.

5.8.8- معايير تقييم الأداء

بعد الانتهاء من تدريب النماذج وضبط معاملات كل منها، جرى تقييم أدائها باستخدام مجموعة الاختبار. استند التقييم على مجموعة من المقاييس الإحصائية الشائعة في مسائل التصنيف الثنائي، شملت الضبط (Accuracy) لقياس نسبة التصنيفات الصحيحة، والدقة (Precision) لتقييم نسبة العينات المصنفة كهجمات والتي كانت هجمات فعلية، والاستدعاء (Recall) لقياس قدرة النموذج على اكتشاف الهجمات، ودرجة F1 (F1-Score) التي تمثل المتوسط التوافقي بين الدقة والاستدعاء، بالإضافة إلى المساحة تحت منحنى ROC (Receiver Operating Characteristic) لقياس قدرة النموذج على التمييز بين الحركة الطبيعية والهجومية بصورة مستقلة عن عتبة التصنيف. من بين هذه المقاييس تم اختيار درجة F1 معياراً رئيسياً للمقارنة بين النماذج، لكونها تحقق توازناً بين الدقة (Precision) والاستدعاء (Recall)، وتوفر تقييماً أكثر موثوقية من الاعتماد على الدقة الإجمالية (Accuracy) وحدها في مسائل كشف الهجمات، حيث نهتم باكتشاف الهجمات - عن طريق Recall - وتقليل الإنذارات الكاذبة (False Positives) - عن طريق Precision -.

6.8.8- تحليل النتائج ومناقشتها

يوضح الجدول التالي نتائج تقييم النماذج باستخدام المقاييس المعتمدة، مع التركيز على درجة F1 بوصفها المعيار الرئيس للمقارنة.

الجدول 10 - نتائج خوارزميات التدريب.

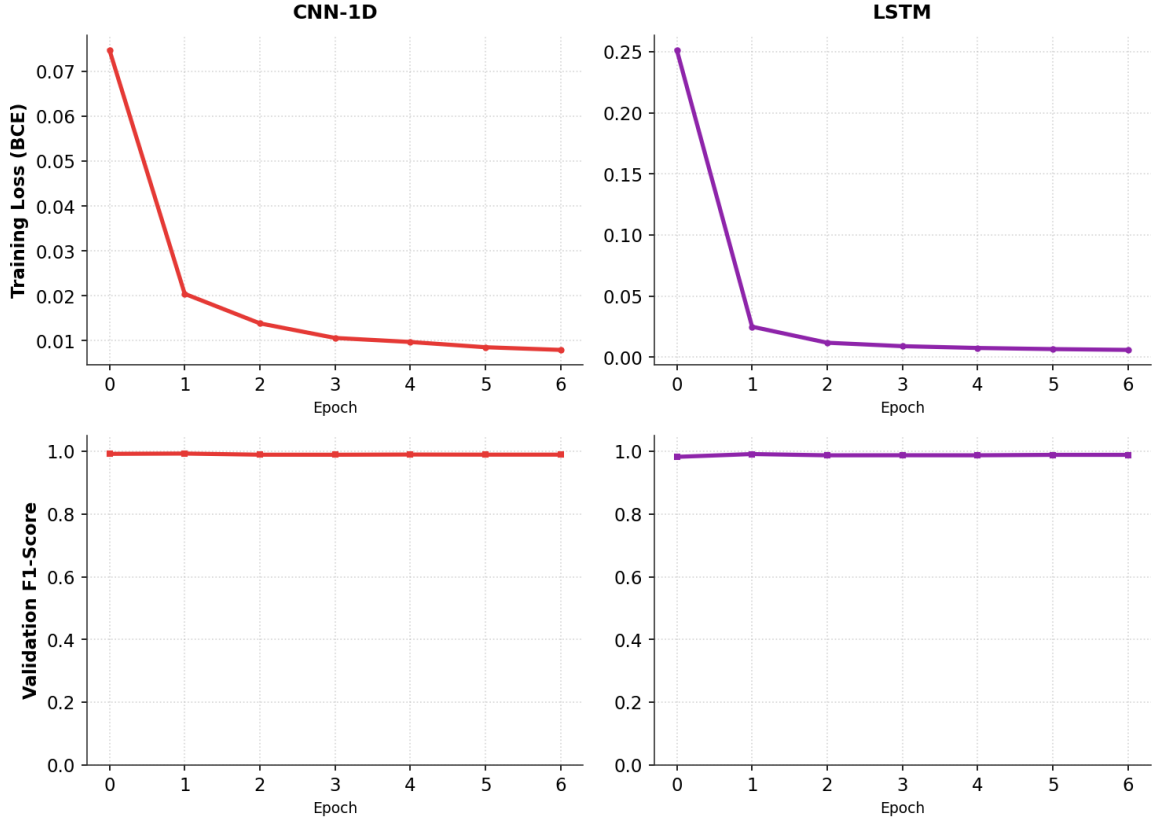
ROC-AUC	F1-Score	Recall	Precision	Accuracy	النموذج
99.92%	97.91%	95.94%	99.95%	98.19%	XGBoost
99.82%	96.55%	93.45%	99.85%	97.05%	LightGBM
99.87%	96.19%	92.85%	99.78%	96.76%	Random Forest
95.96%	95.54%	91.82%	99.56%	96.22%	CNN-1D
98.96%	95.35%	91.42%	99.63%	96.06%	LSTM

تشير النتائج إلى أن جميع النماذج حققت قيمة مرتفعة جداً للدقة تجاوزت 99%، مما يدل على انخفاض عدد الإنذارات الكاذبة (False Positives)، وهو أمر بالغ الأهمية في أنظمة كشف الهجمات لتجنب إصدار تنبيهات غير ضرورية.

في المقابل، ظهر التباين الحقيقي بين النماذج في الاستدعاء ودرجة F1، إذ حقق نموذج XGBoost أعلى قيمة لـ F1 بلغت 97.91%، متفوقاً على جميع النماذج الأخرى. يعكس ذلك قدرة النموذج على تحقيق توازن أفضل بين اكتشاف أكبر عدد ممكن من الهجمات مع المحافظة على انخفاض الإنذارات الكاذبة، مما يجعله النموذج الأكثر ملاءمة لهذه الدراسة. كما تُظهر النتائج تفوق خوارزميات Gradient Boosting، والمتمثلة في XGBoost و LightGBM، على كل من Random Forest ونماذج التعلم العميق.

يمثل الشكل 17 منحنيات التدريب لخوارزميات التعلم العميق.

Training Histories — Epoch-Based Deep Learning Models (CNN-1D, LSTM)
(Random Forest, LightGBM, and XGBoost are ensemble/boosted-tree models without a comparable per-epoch loss curve, so they are excluded here)



الشكل 17- منحنيات تدريب خوارزميات التعلّم العميق.

يوضح الشكل 17 منحنيات التدريب لنموذجي CNN-1D و LSTM، حيث يظهر الانخفاض المستمر في قيمة الخسارة بالتزامن مع استقرار درجة F1 على مجموعة التحقق عند قيم مرتفعة، مما يشير إلى تقارب النموذجين واستقرار عملية التدريب. اقتصرَت منحنيات التدريب على نموذجي التعلّم العميق المستخدمين لأنهما وحدهما يُدرَّبان بطريقة تكرارية عبر حقبة (Epochs) باستخدام انحدار التدرّج (Gradient Descent)، مما يتيح تتبّع تطور الفقدان ودرجة F1 عبر الزمن. أما النماذج الشجرية (Random Forest, LightGBM, XGBoost) فلا تتبع مفهوم الحقبة، إذ تُبنى فيها الأشجار دفعة واحدة أو تباعاً دون تسجيل تاريخ تطوّر الأداء عبر التكرارات، لذا اكتُفي بعرض نتائجها النهائية فقط دون منحنى تدريجي.

7.8.8- اختيار الخوارزمية الأنسب

استناداً إلى نتائج التقييم، تبين أن خوارزمية XGBoost حققت أفضل أداء بين جميع النماذج المدروسة، إذ سجلت أعلى القيم في معظم مؤشرات الأداء، ولا سيما درجة F1 التي اعتُمدت معياراً رئيسياً للمقارنة، بالإضافة إلى تفوقها في كل من الضبط، والدقة، والاستدعاء، والمساحة تحت منحنى ROC. يشير ذلك إلى قدرتها العالية على تحقيق توازن فعال بين اكتشاف الهجمات وتقليل الإنذارات الكاذبة، مع المحافظة على زمن تدريب مناسب.

بالنسبة إلى نماذج CNN-1D و LSTM، فقد أظهرت نتائج جيدة، إلا أنها لم تتفوق على نماذج Boosting في هذه الدراسة. يمكن تفسير ذلك بطبيعة البيانات المستخدمة، إذ تعتمد الدراسة على سمات إحصائية جدولية (Tabular Features)، وهي من أنواع البيانات التي تحقق معها خوارزميات Boosting أداءً أفضل مقارنةً بالشبكات العصبونية العميقة، التي تُظهر كفاءة أكبر عند التعامل مع البيانات ذات البنية الزمنية أو المكانية الواضحة.

بناءً على ذلك، اعتمد نموذج XGBoost بوصفه النموذج الأنسب لهذه الدراسة، نظراً لتحقيقه أفضل توازن بين دقة التصنيف، والقدرة على اكتشاف الهجمات، مما يجعله الخيار الأكثر ملاءمة للاعتماد عليه في الدراسات المستقبلية أو في تطبيقات كشف الهجمات المشابهة.

استعرض هذا الفصل الجوانب التنفيذية لنظام جدار النار المبني على SDN، موضحاً آليات تنجيز وحداته البرمجية والقرارات التقنية المتبعة في تطويره، بالإضافة إلى الدراسة المقارنة لخوارزميات الذكاء الاصطناعي واختيار النموذج الأنسب لتصنيف الهجمات. يُخصّص الفصل التالي لعرض نتائج الاختبارات العملية ومناقشة أداء النظام والتحقق من مدى تحقيقه للأهداف والمتطلبات المحددة.

الفصل التاسع

اختبارات النظام ومناقشة النتائج

يوضح هذا الفصل الاختبارات التي تم تطبيقها للتأكد من تلبية النظام للمتطلبات.

1.9- اختبار الوحدات (Units Testing)

اختبار الوحدات هو أسلوب اختبار يركز على التحقق من صحة كل وحدة برمجية بشكل مستقل تماماً عن بقية مكونات النظام، إذ يُختبر كل دالة أو مجموعة دوال بمعزل عن Ryu و Flask والشبكة. تكمن أهميته في اكتشاف الأخطاء في منطق الكود مبكراً قبل ربط المكونات ببعضها، مما يُقلل تكلفة الإصلاح ويضمن موثوقية كل وحدة على حدة.

نُفذت الاختبارات في ملف مستقل (مكتوب بلغة Python)، وتنقسم إلى خمس مجموعات:

- **TestDatabaseInit**: للتحقق من إنشاء الجداول الثلاث - rules و logs و rule_stats - بشكل صحيح، ومن أن إنشاء الجداول مرتين لا يُسبب أي خطأ.
- **TestRuleCRUD**: مجموعة دوال تُدرج قاعدة في قاعدة المعطيات وتقرأها وتتحقق من قيمها، ودوال أخرى تُعدّل القاعدة وتتحقق من تغيير حقوقها، ودوال تحذف القاعدة وتتحقق من زوالها. كما نتحقق من أن الحذف يُزيل سجل الإحصائيات المرتبط تلقائياً بفضل علاقة CASCADE.
- **TestLogging**: دوال تُدرج أحداثاً أمنية في جدول logs وتقرأها، وتتحقق من صحة تعيين أسماء الأحداث (Rule Added و Traffic Blocked وغيرها)، وتتحقق من آلية التنظيف التلقائي عند تجاوز الحد الأقصى البالغ 10,000 إدخال.
- **TestRuleStats**: تم كتابة دوال من صحة تتبع عداد الرزم المحجوبة في جدول rule_stats، وتتحقق من أن القيمة تُحدّث بشكل صحيح عند استطلاع إحصائيات التدفق.
- **TestMatchFields**: دوال للتحقق من صحة تحويل معلمات القاعدة إلى حقول مطابقة OpenFlow، تشمل التحقق من وجود eth_type=0x0800 دائماً، وتعيين ip_proto=6 ل TCP و ip_proto=17 ل UDP و ip_proto=1 ل ICMP، وعدم إضافة حقول المنافذ عند استخدام ICMP.

Result 	Test
Passed in 30 ms	test_unit.py::TestDatabaseInit::test_init_idempotent
Passed in 22 ms	test_unit.py::TestDatabaseInit::test_logs_table_created
Passed in 21 ms	test_unit.py::TestDatabaseInit::test_rule_stats_table_created
Passed in 20 ms	test_unit.py::TestDatabaseInit::test_rules_table_created
Passed in 24 ms	test_unit.py::TestRuleCRUD::test_allow_rule_default_priority
Passed in 22 ms	test_unit.py::TestRuleCRUD::test_block_rule_default_priority
Passed in 29 ms	test_unit.py::TestRuleCRUD::test_delete_cascades_to_stats
Passed in 27 ms	test_unit.py::TestRuleCRUD::test_delete_removes_rule
Passed in 40 ms	test_unit.py::TestRuleCRUD::test_insert_multiple_rules
Passed in 20 ms	test_unit.py::TestRuleCRUD::test_insert_returns_positive_id
Passed in 21 ms	test_unit.py::TestRuleCRUD::test_insert_rule_creates_stats_entry
Passed in 20 ms	test_unit.py::TestRuleCRUD::test_load_after_insert_returns_one_rule
Passed in 18 ms	test_unit.py::TestRuleCRUD::test_load_empty_returns_empty_list
Passed in 29 ms	test_unit.py::TestRuleCRUD::test_rules_ordered_by_priority_desc
Passed in 31 ms	test_unit.py::TestRuleCRUD::test_update_changes_fields
Passed in 74 ms	test_unit.py::TestLogging::test_cleanup_keeps_max_logs
Passed in 45 ms	test_unit.py::TestLogging::test_clear_logs_empties_table
Passed in 29 ms	test_unit.py::TestLogging::test_insert_log_persists
Passed in 25 ms	test_unit.py::TestLogging::test_load_empty_logs
Passed in 21 ms	test_unit.py::TestLogging::test_log_event_block
Passed in 24 ms	test_unit.py::TestLogging::test_log_event_rule_added
Passed in 21 ms	test_unit.py::TestLogging::test_log_event_rule_deleted
Passed in 19 ms	test_unit.py::TestLogging::test_log_event_rule_modified
Passed in 71 ms	test_unit.py::TestLogging::test_log_limit_parameter
Passed in 25 ms	test_unit.py::TestRuleStats::test_nonexistent_rule_returns_zero
Passed in 45 ms	test_unit.py::TestRuleStats::test_stat_increments_correctly
Passed in 35 ms	test_unit.py::TestRuleStats::test_update_and_retrieve_stat
Passed in 20 ms	test_unit.py::TestMatchFields::test_dst_ip_included
Passed in 19 ms	test_unit.py::TestMatchFields::test_empty_src_not_included
Passed in 22 ms	test_unit.py::TestMatchFields::test_eth_type_always_present
Passed in 22 ms	test_unit.py::TestMatchFields::test_icmp_has_no_ports
Passed in 22 ms	test_unit.py::TestMatchFields::test_icmp_proto_number
Passed in 27 ms	test_unit.py::TestMatchFields::test_src_ip_included
Passed in 18 ms	test_unit.py::TestMatchFields::test_tcp_dport_included

الشكل 18- نتائج اختبارات الوحدات.

Passed in 25 ms	test_unit.py::TestMatchFields::test_tcp_proto_number
Passed in 26 ms	test_unit.py::TestMatchFields::test_tcp_sport_included
Passed in 25 ms	test_unit.py::TestMatchFields::test_udp_dport_included
Passed in 23 ms	test_unit.py::TestMatchFields::test_udp_proto_number
Passed in 23 ms	test_unit.py::TestMatchFields::test_udp_sport_included

الشكل 19- نتائج اختبارات الوحدات (تكملة).

أظهرت نتائج اختبارات الوحدات (الشكلين 18 و 19) نجاح جميع الوحدات المختبرة دون تسجيل أي حالات فشل، مما يؤكد سلامة تنفيذ الوظائف الأساسية لكل وحدة واستيفائها للمتطلبات المحددة.

2.9- اختبار التكامل (Integration Testing)

اختبار التكامل هو أسلوب اختبار يتحقق من صحة تعاون مكوّنات النظام المختلفة معاً، إذ يُطلب كل اختبار اتصالاً فعلياً بين Flask و Ryu و SQLite في آنٍ واحد. يكشف هذا النوع من الاختبارات عن الأخطاء التي لا تظهر في اختبارات الوحدات كأخطاء التواصل بين المكوّنات وعدم تطابق البيانات المرسل والمستقبلة.

تُنفَّذ الاختبارات في ملف مستقل وتنقسم إلى أربع مجموعات:

- **TestConnectivity**: تم كتابة دوال تتحقق من أن Flask يستجيب على المنفذ 5000 وأن Ryu يستجيب على المنفذ 8080، وأن كليهما يُعيدان بيانات JSON صحيحة.
- **TestRuleLifecycle**: مجموعة من الدوال تُضيف قواعد بمعلومات مختلفة (TCP و UDP و ICMP وعنوان IP الوجهة ومنافذ المصدر والوجهة وقواعد السماح) وتتحقق من ظهورها في نتيجة /api/rules، ودالة تُعدّل قاعدة وتتحقق من تغيير قيمها، ودالة تحذف قاعدة وتتحقق من اختفائها.
- **TestLoggingIntegration**: مجموعة من الدوال تُضيف قاعدة وتتحقق من أن حدث "Rule Added" ظهر في السجلات، ودوال تحذف وتُعدّل وتتحقق من ظهور "Rule Deleted" و "Rule Modified" على التوالي.
- تعدد القواعد المتزامنة: تم كتابة دالة تُضيف ثلاث قواعد بروتوكولات مختلفة وتتحقق من أن الثلاثة تعايشت معاً دون أي تعارض أو فقدان.

Result ▲	Test
Passed in 14 ms	test_integration.py::TestConnectivity::test_flask_reachable
Passed in 8 ms	test_integration.py::TestConnectivity::test_get_logs_returns_json_list
Passed in 6 ms	test_integration.py::TestConnectivity::test_get_rules_returns_json_list
Passed in 4 ms	test_integration.py::TestConnectivity::test_ryu_reachable
Passed in 409 ms	test_integration.py::TestRuleLifecycle::test_add_allow_rule
Passed in 397 ms	test_integration.py::TestRuleLifecycle::test_add_icmp_block_rule
Passed in 399 ms	test_integration.py::TestRuleLifecycle::test_add_rule_with_dport
Passed in 403 ms	test_integration.py::TestRuleLifecycle::test_add_rule_with_dst_ip
Passed in 416 ms	test_integration.py::TestRuleLifecycle::test_add_rule_with_sport
Passed in 454 ms	test_integration.py::TestRuleLifecycle::test_add_tcp_block_rule
Passed in 391 ms	test_integration.py::TestRuleLifecycle::test_add_udp_block_rule
Passed in 439 ms	test_integration.py::TestRuleLifecycle::test_delete_rule_removes_it
Passed in 729 ms	test_integration.py::TestRuleLifecycle::test_modify_rule_updates_fields
Passed in 649 ms	test_integration.py::TestRuleLifecycle::test_multiple_rules_coexist
Passed in 1 s	test_integration.py::TestLoggingIntegration::test_add_rule_creates_rule_added_log
Passed in 1 s	test_integration.py::TestLoggingIntegration::test_clear_logs_empties_log_table
Passed in 910 ms	test_integration.py::TestLoggingIntegration::test_delete_rule_creates_rule_deleted_log
Passed in 1 s	test_integration.py::TestLoggingIntegration::test_modify_rule_creates_rule_modified_log

الشكل 20- نتائج اختبارات التكامل.

يُظهر الشكل 20 نتائج تشغيل ملف اختبار التكامل، حيث نجحت جميع الاختبارات التي تغطي دورة حياة القواعد الكاملة من الإضافة إلى التعديل إلى الحذف، مما يُثبت صحة التواصل بين Flask و Ryu و SQLite في آنٍ واحد.

3.9- الاختبارات الأمنية (Security Testing)

الاختبارات الأمنية هي أسلوب اختبار يتحقق من قدرة النظام على التصدي للمدخلات الخبيثة والهجمات الشائعة دون التعطل أو الكشف عن ثغرات. تشمل هذه الاختبارات محاولات حقن SQL وهجمات XSS والمدخلات المشوهة وحالات الحدود الطرفية.

نُفذت الاختبارات في ملف مستقل وتنقسم إلى أربع مجموعات:

- **TestInputValidation**: تم كتابة مجموعة من الدوال ترسل عنوان IP غير صالح (999.999.999.999) وجسم POST فارغ وحقولاً مفقودة وقيم أولوية سالبة وقيم حدودية (0 و 65535)، وتتحقق في كل حالة من أن النظام لا يتعطل ولا يزال يستجيب بشكل طبيعي.
- **TestSQLInjection**: كتبنا دوال ترسل محاولات SQL Injection في حقول src و dst و proto تشمل:
 - 'DROP TABLE rules' لمحاولة حذف جدول القواعد
 - 'DELETE FROM rules' لمحاولة حذف جميع القواعد
 - 'SELECT * FROM logs' لمحاولة تسريب بيانات السجلات.وتتحقق في كل حالة من أن الجداول لا تزال موجودة وأن النظام يستجيب بشكل سليم.
- **TestXSS**: مجموعة من الدوال ترسل محاولات XSS في حقول الإدخال تشمل `<script>alert('xss')</script>` وعلامات `<img>` الخبيثة، وتتحقق من أن النظام لا يتعطل وأن قاعدة المعطيات تظل سليمة.
- **TestAPIRobustness**: كتبنا دوال ترسل طلبات بـ Content-Type خاطيء، وتحذف وتُعدّل قواعد غير موجودة، وتتحقق من أن النظام يتعامل مع كل هذه الحالات بأمان.

Result ▲	Test
Passed in 509 ms	test_security.py::TestInputValidation::test_all_endpoints_return_valid_json
Passed in 396 ms	test_security.py::TestInputValidation::test_negative_priority_handled
Passed in 388 ms	test_security.py::TestInputValidation::test_priority_boundary_high
Passed in 487 ms	test_security.py::TestInputValidation::test_priority_boundary_low
Passed in 354 ms	test_security.py::TestInputValidation::test_system_survives_empty_body
Passed in 389 ms	test_security.py::TestInputValidation::test_system_survives_invalid_ip
Passed in 388 ms	test_security.py::TestInputValidation::test_system_survives_missing_fields
Passed in 397 ms	test_security.py::TestSQLInjection::test_sql_injection_in_dst_field
Passed in 392 ms	test_security.py::TestSQLInjection::test_sql_injection_in_proto_field
Passed in 396 ms	test_security.py::TestSQLInjection::test_sql_injection_in_src_field
Passed in 395 ms	test_security.py::TestSQLInjection::test_union_based_injection
Passed in 406 ms	test_security.py::TestXSS::test_xss_img_tag_in_dst
Passed in 392 ms	test_security.py::TestXSS::test_xss_in_src_field
Passed in 14 ms	test_security.py::TestAPIRobustness::test_delete_nonexistent_rule
Passed in 13 ms	test_security.py::TestAPIRobustness::test_modify_nonexistent_rule
Passed in 12 ms	test_security.py::TestAPIRobustness::test_wrong_content_type

الشكل 21- نتائج الاختبارات الأمنية.

أظهرت نتائج الاختبارات الأمنية (الشكل 21) صمود النظام أمام جميع محاولات الحقن والمدخلات الخبيثة دون تعطل، مما يُثبت أن استخدام Parameterized Queries في SQLite يحمي النظام من هجمات SQL Injection بشكل تلقائي.

4.9- اختبارات الحمل (Load Testing)

اختبارات الحمل هي أسلوب اختبار يتحقق من قدرة النظام على التعامل مع حجم طبيعي إلى مرتفع من الطلبات المتزامنة والمتسلسلة دون تدهور في الأداء أو حدوث أخطاء. يُساعد هذا النوع على تحديد سلوك النظام تحت ظروف الاستخدام الواقعي.

تُنفذ الاختبارات في ملف مستقل وتنقسم إلى ثلاث مجموعات:

- **TestSequentialLoad**: تم كتابة دالة تُضيف 20 قاعدة متسلسلة وتتحقق من نجاحها جميعاً دون أي خطأ، ودالة أخرى تُرسل 50 طلب GET متسلسلاً على `/api/rules` و `/api/logs` وتتحقق من أن جميعها تُعيد `HTTP 200`، ودالة تُنفذ دورة إضافة وحذف لنفس القاعدة 10 مرات متتالية وتتحقق من أن النظام ينتهي بصفر قواعد (تم اختيار 20 قاعدة متسلسلة بسبب عدم توفر ظروف تشغيلية حقيقية، حيث يُشير الرقم 20 إلى عدد القواعد التي قد يضيفها مدير النظام والذي قد يختلف بتوفر بيئة تشغيل واقعية).

- **TestConcurrentLoad**: كتبنا دالة تُطلق 10 نياسب متزامنة تُضيف قواعد في آنٍ واحد وتتحقق من نجاح جميعها، ودالة تُطلق 3 نياسب للقراءة و5 أخرى للكتابة في نفس الوقت وتتحقق من عدم حدوث أي تعارض، ودالة تُطلق 20 نيسباً متزامناً لطلبات GET وتتحقق من نجاح جميعها.
- **TestResponseTime**: مجموعة من الدوال تقيس زمن استجابة عمليات GET و POST و DELETE وتتحقق من أن كل عملية تستجيب في أقل من ثانيتين، ودالة تحسب متوسط زمن الاستجابة على 10 طلبات متتالية وتتحقق من أنه يقل عن ثانية واحدة.

Result ▲	Test
Passed in 662 ms	test_load.py::TestSequentialLoad::test_50_rapid_get_logs
Passed in 852 ms	test_load.py::TestSequentialLoad::test_50_rapid_get_rules
Passed in 1 s	test_load.py::TestSequentialLoad::test_add_20_rules_sequentially
Passed in 785 ms	test_load.py::TestSequentialLoad::test_add_delete_cycle_10_times
Passed in 689 ms	test_load.py::TestConcurrentLoad::test_10_concurrent_rule_additions
Passed in 424 ms	test_load.py::TestConcurrentLoad::test_20_concurrent_get_requests
Passed in 632 ms	test_load.py::TestConcurrentLoad::test_concurrent_reads_and_writes
Passed in 421 ms	test_load.py::TestResponseTime::test_add_rule_under_2s
Passed in 433 ms	test_load.py::TestResponseTime::test_average_response_time_over_10_requests
Passed in 420 ms	test_load.py::TestResponseTime::test_delete_rule_under_2s
Passed in 351 ms	test_load.py::TestResponseTime::test_get_rules_under_2s

الشكل 22- نتائج اختبارات الحمل.

نستنتج من الشكل 22 أن النظام نجح في التعامل مع جميع سيناريوهات الحمل المتسلسل والمتزامن ضمن حدود زمن الاستجابة المقبولة، مما يُثبت استقراره تحت ظروف الاستخدام الواقعي.

5.9- اختبارات الإجهاد (Stress Testing)

اختبارات الإجهاد هي أسلوب اختبار يدفع النظام إلى ما وراء حدود التشغيل الطبيعية لتحديد نقاط الكسر والتحقق من قدرته على التعافي. تختلف عن اختبارات الحمل في كونها تستخدم أحجاماً أكبر بكثير من الطلبات وعدداً أعلى من النياسب المتزامنة بهدف اكتشاف حدود النظام.

نُفذت الاختبارات في ملف مستقل وتنقسم إلى أربع مجموعات:

- **TestHighVolumeRules**: تم كتابة دالة تُضيف 50 قاعدة متسلسلة وتتحقق من نجاحها جميعاً، وأخرى تتحقق من أن النظام لا يزال يستجيب بشكل طبيعي بعد إضافة 50 قاعدة، ودالة تُضيف 50 قاعدة ثم تحذفها جميعاً وتتحقق من أن النظام ينتهي بصفر قواعد.

- **TestHighVolumeLogs**: دالتين، واحدة تُنفذ 100 دورة إضافة وحذف وتتحقق من أن السجلات تُنشأ بشكل صحيح، والأخرى تتحقق من أن النظام لا يزال يستجيب بشكل طبيعي بعد 100 دورة.
- **TestHighConcurrency**: كتبنا دالة تُطلق 30 نيسباً متزامناً تُضيف قواعد وتتحقق من نجاح جميعها، ودالة تُطلق 30 نيسباً متزامناً لطلبات GET وتتحقق من نجاح جميعها، ودالة تُطلق 10 نياسب للقراءة و10 أخرى للكتابة و5 نياسب للحذف في نفس الوقت وتتحقق من عدم حدوث أي تعارض أو خطأ.
- **TestRecovery**: دالتين، واحدة تُرسل 30 طلباً متزامناً ثم تتحقق من أن النظام لا يزال يستجيب بشكل طبيعي بعد الانتهاء، وأخرى تقيس زمن الاستجابة بعد دفعة إجهاد وتتحقق من أنه لا يتجاوز 3 ثوانٍ.

Result ▲	Test
Passed in 3 s	test_stress.py::TestHighVolumeRules::test_add_50_rules_without_failure
Passed in 3 s	test_stress.py::TestHighVolumeRules::test_add_and_delete_50_rules
Passed in 3 s	test_stress.py::TestHighVolumeRules::test_system_stable_after_50_rules
Passed in 4 s	test_stress.py::TestHighVolumeLogs::test_100_log_add_delete_cycles_create_logs
Passed in 4 s	test_stress.py::TestHighVolumeLogs::test_system_stable_after_100_log_cycles
Passed in 669 ms	test_stress.py::TestHighConcurrency::test_30_concurrent_get_requests
Passed in 1 s	test_stress.py::TestHighConcurrency::test_30_concurrent_rule_additions
Passed in 1 s	test_stress.py::TestHighConcurrency::test_mixed_operations_under_stress
Passed in 2 s	test_stress.py::TestRecovery::test_response_time_after_stress
Passed in 1 s	test_stress.py::TestRecovery::test_system_responds_after_burst

الشكل 23- نتائج اختبارات الإجهاد.

يُظهر الشكل 23 نتائج اختبارات الإجهاد، حيث أثبت النظام مرونته تحت أقصى ظروف التشغيل بما فيها 50 قاعدة متزامنة و30 نيسباً متوازياً، وتعافى بشكل طبيعي بعد كل دفعة إجهاد دون أي تعطل أو فقدان بيانات.

6.9- اختبارات الأداء (Performance Testing)

اختبارات الأداء هي أسلوب اختبار يُقاس ويُحلّل الخصائص الكمية للنظام كزمن الاستجابة والإنتاجية (Throughput) والمئين¹ (Percentiles). تختلف عن اختبارات الحمل في تركيزها على القياس الدقيق بدلاً من التحقق من الصحة فقط، مما يُوفّر أرقاماً موضوعية لتوصيف أداء النظام.

نُفذت الاختبارات في ملف مستقل وتنقسم إلى خمس مجموعات:

¹ قيمة إحصائية نحبرنا بأن نسبة معينة من البيانات تقع تحت هذه القيمة. مثلاً: إذا كان المئين 95 = 500 ms فهذا يعني أن 95% من الطلبات استجابت في أقل من 500 ms، و 5% فقط كانت أبطأ من ذلك. إذا كان المئين 50 = 200 ms فهذا يعني أن نصف الطلبات استجابت في أقل من 200 ms، وهو ما يُسمى الوسيط (Median).

- TestGetRulesPerformance: مجموعة من الدوال تُنفَّذ 20 طلب GET على /api/rules وتحسب المتوسط والمئين 95 وتتحقق من أنهما ضمن الحدود المحددة، ودالة تتحقق من الأداء مع وجود 10 قواعد مُحَمَّلة مسبقاً.
- TestPostRulesPerformance: مجموعة من الدوال تُنفَّذ 20 طلب POST لإضافة قواعد وتحسب المتوسط والمئين 95 لزمن الاستجابة.
- TestDeleteRulesPerformance: دالة تُضيف 20 قاعدة مسبقاً ثم تقيس زمن حذف كل منها وتحسب المتوسط.
- TestThroughput: دالة تُرسل أكبر عدد ممكن من طلبات GET خلال 5 ثوانٍ وتحسب عدد الطلبات في الثانية، ودالة تُطلق 4 نياسب متوازية وتقيس الإنتاجية الكلية.
- TestGetLogsPerformance: تم كتابة دوال تقيس أداء /api/logs بنفس الطريقة وتتحقق من أنه ضمن الحدود المحددة.

مع تعريف حدود أداء واضحة:

- الحد الأقصى لمتوسط زمن الاستجابة: 300 ميلي ثانية.
- الحد الأقصى للمئين 95 (P95) لطلبات GET: 500 ميلي ثانية.
- الحد الأقصى للمئين 95 لطلبات POST: 1000 ميلي ثانية.
- الحد الأدنى للإنتاجية: 5 طلبات في الثانية.

Result ▲	Test
Passed in 455 ms	test_performance.py::TestGetRulesPerformance::test_get_rules_average_response_time
Passed in 485 ms	test_performance.py::TestGetRulesPerformance::test_get_rules_p95_response_time
Passed in 874 ms	test_performance.py::TestGetRulesPerformance::test_get_rules_with_10_existing_rules
Passed in 1 s	test_performance.py::TestPostRulesPerformance::test_add_rule_average_response_time
Passed in 1 s	test_performance.py::TestPostRulesPerformance::test_add_rule_p95_response_time
Passed in 1 s	test_performance.py::TestDeleteRulesPerformance::test_delete_rule_average_response_time
Passed in 5 s	test_performance.py::TestThroughput::test_concurrent_throughput
Passed in 5 s	test_performance.py::TestThroughput::test_get_throughput
Passed in 458 ms	test_performance.py::TestGetLogsPerformance::test_get_logs_average_response_time
Passed in 459 ms	test_performance.py::TestGetLogsPerformance::test_get_logs_p95_response_time

الشكل 24- نتائج اختبارات الأداء.

تُظهر نتائج اختبارات الأداء — كما في الشكل 24 — مع الأرقام الفعلية لمتوسط زمن الاستجابة والمئين 95 وعدد الطلبات في الثانية، حيث حقّق النظام أداءً ضمن الحدود المحددة لجميع عمليات القراءة والكتابة والحذف.

7.9- تغطية المتطلبات الوظيفية من خلال مكونات النظام

يُوضّح هذا القسم العلاقة المباشرة بين كل متطلب وظيفي من متطلبات النظام والمكونات البرمجية التي تُنجزه فعلياً. يُساعد هذا التعيين على التحقق من اكتمال التنجيز وضمان أن كل متطلب مُغطّى بمكوّن محدد قابل للاختبار والتتبع.

الجدول 11- المتطلبات الوظيفية المنجزة.

المكون المنجّر	المتطلب الوظيفي
صفحة Policies + قسم إضافة قاعدة + نافذة التعديل	1. تسجيل القواعد الأمنية عبر واجهة وب
Ryu Controller + دوال دفع القاعدة إلى المبدّلات وتطبيقها	2. تطبيق القواعد على التدفقات الشبكية باستخدام SDN Controller
REST API /firewall/rules/modify + حذف وإعادة تثبيت قاعدة التدفق + دالة تعديل قاعدة المنجزة	3. تعديل القواعد ديناميكياً أثناء التشغيل
صفحة Monitoring + رسومات SVG + جدول Block Report + آلية إحصائيات التدفق	4. مراقبة حركة المرور وتوليد تقارير
صفحة Event Logs + جدول logs في SQLite + دالة إدخال سجل أحداث (Insert Log) إلى قاعدة المعطيات	5. سجل أحداث لمتابعة النشاط الأمني
نظام الأولويات + مطابقة حقول IP والمنافذ والميفاق في OpenFlow	6. دعم الحجب أو السماح بناءً على المصدر والوجهة أو الميفاق

يُبيّن الجدول 11 أن جميع المتطلبات الوظيفية الستة قد غُطّيت بالكامل من خلال مكونات محددة وقابلة للتتبع، إذ يُنجز كل متطلب بتعاون مكوّن الواجهة الأمامية مع المنطق الخلفي في المتحكم وقاعدة المعطيات.

الخاتمة والآفاق المستقبلية

في ختام هذا المشروع، تم بنجاح تصميم وتنفيذ نظام جدار نار ديناميكي مبني على تقنية SDN، قادر على إدارة السياسات الأمنية الشبكية وتطبيقها في الزمن الحقيقي عبر واجهة وب تفاعلية احترافية. لقد أثبت المشروع جدوى توظيف مبدأ فصل مستوي التحكم عن مستوي تمرير البيانات في بناء جدران نار مرنة وقابلة للإدارة المركزية، إذ يتولى متحكم Ryu إدارة جميع المبدلات الافتراضية في آن واحد عبر ميفاق OpenFlow 1.3، مما يُتيح تطبيق السياسات الأمنية وتعديلها فوراً على كامل الشبكة دون الحاجة إلى إعادة تشغيل أي مكوّن.

من خلال اعتماد قاعدة معطيات SQLite لتخزين القواعد الأمنية وسجلات الأحداث بصورة دائمة، تم تجاوز أحد أبرز قيود تطبيقات SDN البحثية المتمثل في فقدان السياسات عند إعادة تشغيل المتحكم. كما أسهمت آلية استطلاع إحصائيات تدفق OpenFlow في رصد حركة السير المحجوبة بدقة عالية متجاوزة إشكالية عدم وصول الرزم المحجوبة إلى المتحكم بعد تثبيت قواعد Drop Flow على المبدلات.

أكدت نتائج الاختبارات الشاملة التي أُجريت على النظام تحقيق جميع المتطلبات الوظيفية المحددة؛ إذ أثبت النظام قدرته على حجب حركة السير بشكل انتقائي دقيق وفق معلمات متعددة تشمل عنوان IP المصدر والوجهة والبروتوكول والمنفذ، مع ضمان استمرار تقديم الخدمة للمستخدمين الشرعيين خلال فترات الهجوم. كما أثبت نظام الأولويات المنجز كفاءته في حل تعارض القواعد بتغليب قرارات الحجب دائماً على قرارات السماح، محققاً مبدأ الأمان التحفظي المطلوب في أنظمة الحماية الشبكية.

بشكل عام، يُقدّم هذا المشروع نموذجاً عملياً متكاملًا يُجسّد إمكانيات تقنية SDN في بناء حلول أمنية شبكية ديناميكية وفعّالة، ويُوفّر أساساً متيناً يمكن البناء عليه مستقبلاً.

وبناءً على النتائج والاستنتاجات التي تم التوصل إليها، يمكن اقتراح مجموعة من التحسينات والتوسعات المستقبلية:

تطوير قدرات جدار النار:

- جدار النار ذو الحالة: تطوير النظام الحالي من جدار نار عديم الحالة إلى جدار نار ذي حالة يتتبع حالة الاتصالات الشبكية، مما يُتيح التمييز بين حركة السير المبادر بها وحركة السير الاستجابية وتطبيق سياسات أمنية أكثر دقة وتعقيداً.
- القواعد الزمنية (Time-based Rules): إضافة دعم للقواعد الأمنية المقيّدة زمنياً، كحجب حركة سير معينة خارج ساعات العمل الرسمية أو تفعيل سياسات مختلفة بحسب وقت اليوم أو يوم الأسبوع.

- القواعد التكيفية (Adaptive Rules): تطوير آلية لتحديث القواعد تلقائياً بناءً على أنماط حركة السير المرصودة، كالحجب التلقائي لعناوين IP التي تتجاوز حداً معيناً من الطلبات في وحدة زمن محددة.

تحسين نظام المراقبة والتحليل:

- المراقبة في الزمن الحقيقي (Real-time Monitoring): تطوير آلية المراقبة من الاستطلاع الدوري كل 5 ثوانٍ إلى نظام إشعارات فورية يعتمد على WebSocket لتحديث واجهة الوب لحظةً بلحظة دون الحاجة إلى استطلاع دوري.
- تحليل حركة السير: إضافة وحدة تحليل إحصائي لحركة السير تُوفّر رؤية تفصيلية حول أنماط الاستخدام والتهديدات المحتملة، مع إمكانية تصدير التقارير بصيغ مختلفة.
- نظام التنبيهات: تنجز نظام إشعارات يُنبئ مدير النظام فور اكتشاف أنماط هجوم مشبوهة كارتفاع مفاجئ في عدد الحزم المحجوبة من عنوان IP معين.

تكامّل الذكاء الصناعي:

- نظام كشف التسلل (IDS): دمج نموذج تعلم آلي مُدرّب على مجموعة بيانات لاكتشاف أنماط الهجمات الشبكية تلقائياً وإضافة قواعد حجب ديناميكية استجابةً للتهديدات المكتشفة في الزمن الحقيقي.
- التحليل السلوكي: توظيف خوارزميات تعلّم الآلة لبناء ملفات سلوكية للمضيفين في الشبكة واكتشاف الانحرافات غير الطبيعية التي قد تُشير إلى اختراق أمني.

تحسين قابلية التوسع والنشر:

- دعم متحكمات متعددة: تطوير النظام لدعم نشر متحكمات Ryu متعددة في تكوين عالي التوافر (Highly Available) لضمان استمرارية الخدمة في حال فشل أحد المتحكمات.
- واجهة برمجية موسّعة: توسيع واجهة REST API لدعم تكاملات خارجية مع أنظمة إدارة الأمن المعلوماتي مثل SIEM وأنظمة الاستجابة للحوادث الأمنية.
- دعم IPv6: توسيع النظام الحالي المبني على IPv4 ليدعم ميفاق IPv6 استعداداً لبيئات الشبكات الحديثة.

تحسين تجربة المستخدم:

- لوحة تحكم متقدمة: إضافة رسوم بيانية تفاعلية أكثر تفصيلاً تشمل خرائط حرارية لحركة السير وجداول زمنية تفصيلية للأحداث الأمنية.

- النسخ الاحتياطي والاستعادة: تنجيز آلية لتصدير القواعد الأمنية واستيرادها، مما يُتيح نقل السياسات بين بيئات مختلفة أو استعادتها بعد الكوارث.

تفتح هذه الآفاق المجال لمزيد من البحث والتطوير، وتحويل النظام الحالي إلى منصة حماية شبكية أكثر ذكاءً وشمولاً، قادرة على مواجهة التهديدات الأمنية المتطورة في بيئات الشبكات الحديثة.

المراجع

- [1] Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing*. NIST Special Publication 800-145.
- [2] Scarfone, K., & Hoffman, P. (2009). *Guidelines on Firewalls and Firewall Policy*. NIST Special Publication 800-41 Revision 1.
- [3] Kreutz, D., Ramos, F. M. V., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., & Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1), 14–76.
- [4] Albu-Salih, A. T. (2022). Performance Evaluation of Ryu Controller in Software Defined Networks. *Journal of Al-Qadisiyah for Computer Science and Mathematics*, 14(1), 1–7.
- [5] Jayawardena, C., Chen, J., Bhalla, A., & Bui, L. (2025). Comparative Analysis of POX and RYU SDN Controllers in Scalable Networks. *International Journal of Computer Networks and Communications*, 17(2), 35–51.
- [6] Shruthikaa, S., Thanushree, P., & Gunavathie. (2023). SDN-based Firewall using Machine Learning.
- [7] Chowdhary, A., Huang, D., Alshamrani, A., Sabur, A., Kang, M., Kim, A., & Velazquez, A. (2018). SDFW: SDN-based Stateful Distributed Firewall. *arXiv preprint arXiv:1811.00634*.
- [8] Teng, L., Hung, C. H., & Wen, C. H. P4SF: A High-Performance Stateful Firewall on Commodity P4-Programmable Switch.
- [9] Abboud, A. (2021). *Efficient Rules Management Algorithms in Software Defined Networking*. Ph.D. thesis, University of Lorraine.
- [10] Hu, H., Han, W., Ahn, G. J., & Zhao, Z. (2014). FlowGuard: Building Robust Firewalls for Software-Defined Networks. In *Proceedings of the 2014 ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking (HotSDN)*, pp. 97–102.
- [11] <https://www.kaggle.com/datasets/solarmainframe/ids-intrusion-csv>